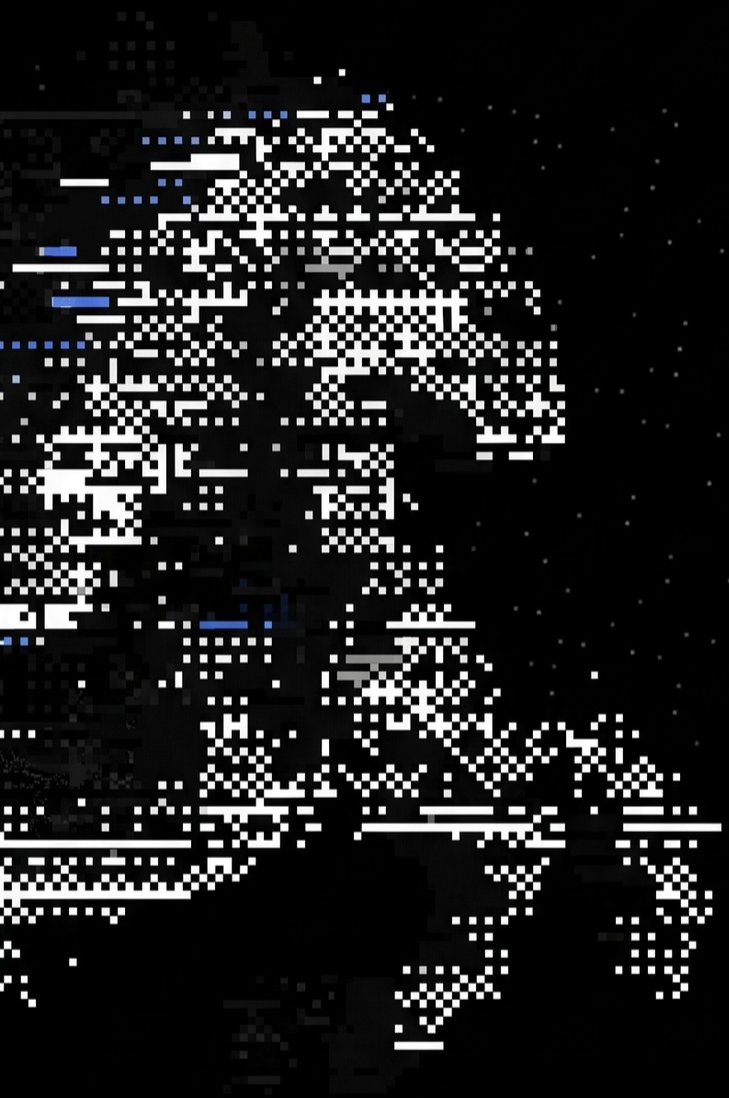
IA



Stack de desarrollo de Software con IA

Qué uso, por qué lo uso y cómo funciona en la práctica — explicado de la mano

Guía práctica para programadores

Autor: Jesús Valverde

ZCode · GLM · MCP · Engram · Scrum · Skills

Octubre 2026

Indice

Índice (clic derecho y «Actualizar campos» para refrescar).

Prefacio

Este libro empezó siendo otra cosa: un documento de 7 páginas que le escribí a un amigo programador para explicarle con qué trabajo todos los días. Él me había preguntado por las herramientas de inteligencia artificial que uso para desarrollar software, y me di cuenta de que la respuesta honesta no cabía en una plática: las herramientas son la parte fácil de contar, y la parte que de verdad importa es cómo se conectan entre sí. Dejé el documento crecer hasta que dejó de ser mensaje y se volvió libro.

Aquí está esa versión crecida. Recorre, pieza por pieza, el stack completo con el que desarrollo software con agentes de IA: la aplicación donde vivo el agente, los modelos que lo piensan, el archivo de reglas que lo domestica, la memoria que lo recuerda todo, el protocolo que le da manos nuevas, los oficios que le enseño y la forma en que dirijo el trabajo cuando el que teclea código ya no soy yo. Cada pieza se explica desde cero, con un ejemplo real de mi práctica, porque el libro se escribió para leerse de corrido.

Antes de que sigas, un aviso de lo que no vas a encontrar. No hay comparativas de mercado ni ranking de herramientas del mes: cuando menciono dos apps es porque uso las dos, y digo en qué se me hace que cada una sirve. No hay promesas de productividad mágica ni recetas para «10x tu output». Y no hay una sola afirmación que no pueda respaldar con mi experiencia o con una fuente citada, hasta donde las cosas de este mundo se dejan respaldar.

El trato didáctico es simple: asumo que sabes programar, que usas terminal y control de versiones, y nada más. Todo lo demás (qué es un modelo, qué es un token, qué significa que un modelo «razona») se explica en su capítulo, desde cero, la primera vez que aparece. Si ya conoces algún tema, sáltalo sin culpa: los capítulos están pensados para sostenerse solos.

Puedes leerlo corrido, como el relato de cómo armé todo esto, o entrar directo al capítulo de la pieza que te interese y salir con lo tuyo. La Parte I abre con lo que está en juego: el miedo profesional, el vibe coding y mi propia línea del tiempo de cómo llegué aquí; después pone los fundamentos para que cualquiera que codea pueda seguir el resto. La Parte II presenta las piezas del stack, una por capítulo. La Parte III muestra cómo se dirigen entre sí: especificaciones, agentes que se revisan entre ellos, tests que deciden, y el diseño cerrando el ciclo. El último capítulo condensa todo en el mínimo viable para que armes el tuyo.

Una confesión para cerrar. El documento original lo mantenía el propio stack: yo lo dictaba, y el agente lo redactaba, lo editaba y lo daba de alta en el repositorio. Este libro sigue la misma ruta, con más pasadas de edición encima. Si en algún momento te suena a que quien lo escribió conoce el tema por dentro, es porque lo conoce desde los dos lados del teclado.

Jesús Valverde

Octubre de 2026

Parte I. Fundamentos

Capítulo 1. Vibe coding, el miedo y tu nuevo papel

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Explicar qué es el vibe coding, qué parte del desarrollo cubre ya y dónde queda su techo.
2. Argumentar por qué el software profesional sigue necesitando gente técnica, con las dos razones de fondo: experiencia y responsabilidad.
3. Describir tu nuevo papel como administrador de agentes y decidir qué aprender (y qué solo conocer) sin caer en el FOMO.

El miedo con nombre

Hablemos del elefante. Si llevas unos años programando, es probable que en algún momento te haya visitado el mismo pensamiento que me visitó a mí: esto avanza tan rápido que me voy a quedar atrás. Aparecen agentes, protocolos, CLI, bases de datos vectoriales, un modelo nuevo cada mes, y con cada anuncio la pregunta se agranda: ¿para qué me necesitan a mí?

El miedo es real y no te lo voy a quitar con optimismo barato. Te lo voy a quitar con mecánica: mirar con precisión qué parte del trabajo ya hace la IA sin ti, qué parte no puede hacer sin ti, y en qué se convierte tu valor cuando el que teclea deja de ser tú. Ese es el mapa de este capítulo, y su conclusión adelantada cabe en una línea: el software no murió; cambió de forma, y tu lugar en él se corre del teclado a la dirección.

Antes y ahora: lo que era el trabajo

Hace unos años, desarrollar una aplicación se repartía en tres grandes cajones: frontend, backend y base de datos. Cada cajón tenía sus especialistas, sus años de aprendizaje y su demanda laboral, y buena parte del trabajo profesional consistía en escribir y mantener el código de esos tres cajones. Las empresas grandes siguen necesitando esa separación, y la seguirán necesitando.

Lo que cambió es el piso. La aplicación «básica» de toda la vida (una interfaz, una API, una base de datos debajo) hoy la genera un modelo en minutos. Ese tipo de sistema era el pan de cada día de muchísimo trabajo de desarrollo, y ese pan ya se hornea solo. De aquí sale el miedo con nombre técnico: vibe coding.

Vibe coding, definido desde cero

Vibe coding es programar pidiéndole la aplicación a la IA y aceptando lo que trae sin revisarlo a fondo. El término lo acuñó Andrej Karpathy (ex de OpenAI y Tesla) en febrero de 2025, describiendo un modo de trabajo en el que te entregas a las vibras: aceptas todo, ni lees el código, y olvidas que el archivo existe (Karpathy, 2025).

Y conviene decirlo claro: el vibe coding funciona. Hay más software produciéndose hoy que en cualquier momento previo, y buena parte lo generan personas que no saben programar: gente de otros rubros que paga su suscripción, describe lo que quiere, y entrega aplicaciones funcionales a clientes reales. Eso ya no es una predicción; es el estado de las cosas a octubre de 2026.

Que un cliente sin formación técnica pueda vibe-codear su sistema interno de altas y bajas significa exactamente esto: la parte mecánica del desarrollo se comoditizó. Si tu valor profesional era escribir la API que cualquiera describe en una oración, ese valor se mudó de casa. La pregunta que decide tu carrera es si tienes algo más que esa parte mecánica.

El techo del vibe coding

El problema del vibe coding se parece al problema de un crédito: no aparece al contratarlo, aparece cuando crece la deuda. Mientras la aplicación es chica y la usa poca gente, ir a ciegas funciona. Cuando el proyecto crece, las ciegas se pagan.

Crecer, en software, significa cosas concretas. La base de datos se satura y hay que optimizar consultas o migrar el motor, y esa decisión tomada a ciegas se convierte en una reescritura completa. El sistema tiene que hablar con software viejo que nadie va a actualizar, con protocolos propios que no se parecen a ninguna API moderna. Hay que escalar, cambiar de nube, depurar en producción a las tres de la mañana. En cada una de esas situaciones, el que no entiende lo que hay debajo no puede ni diagnosticar ni decidir: solo puede volver a pedirle al modelo que adivine, y pagar la adivinanza.

El vibe coding cubre la base de la aplicación. Lo que no cubre es la parte donde la aplicación deja de ser base, y esa parte es a la que llaman los clientes cuando el dinero es real.

Por qué sigue haciendo falta un profesional

Dos razones de fondo, y ninguna es sentimental.

La primera es experiencia. Un profesional trae decisiones de diseño que la IA no inventa: sabe qué arquitectura va a doler en seis meses, qué librería es mantenida y cuál es un hoyo, qué parte del requisito el cliente no dijo pero va a necesitar. El modelo continúa ese criterio; no lo sustituye. Cuando le pides a la IA que «siga» un diseño bien pensado, vuela. Cuando le pides que adivine el diseño, construye castillos de arena muy bonitos.

La segunda es responsabilidad. Alguien tiene que responder por lo que el sistema hace. Si el agente borra registros de la base de producción, la pregunta de la empresa no es «qué modelo era», sino «quién autorizó esto», y la respuesta tiene nombre y apellidos o no hay contrato. Las herramientas lo dicen sin rodeos en sus términos: tú corres el riesgo. La empresa que lava las manos necesita un par de manos que no las lave, y esas manos son técnicas o no son.

Experiencia y responsabilidad son, además, exactamente lo que el vibe coder sin formación no puede comprar con suscripciones. Por eso el miedo se cura con profundidad, no con velocidad: mientras más abajo entiendas el sistema, más arriba llega el trabajo que te necesita.

Tu nuevo papel: administrador de agentes

Ahora sí, el papel nuevo. Hace algunos años, un desarrollador con experiencia ya había dejado de escribir todo el código: planeaba el proyecto, lo partía en tareas, y delegaba en desarrolladores más juniors, revisando lo que le traían. El trabajo era administrar personas y criterio, con el teclado como herramienta ocasional.

Ese mismo papel existe hoy, con los juniors cambiados por agentes. Yo delego tareas a agentes especializados: uno explora el código, otro implementa, otro revisa seguridad, otro corre los tests. Les doy contexto, les fijo criterios, verifico lo que devuelven y decido qué entra y qué se rehace. Mi día se parece más al de un administrador de proyectos técnicos que al de un tecleador, y el conocimiento técnico pesa justo donde más vale: en las decisiones.

Dale la vuelta al miedo con este lente. El agente que «te quita el trabajo» es el mismo que multiplica el alcance de quien sabe dirigirlo. Lo que desaparece es el puesto de escribir el CRUD; lo que aparece es el puesto de administrar el enjambre que escribe cien CRUDs. La Parte II de este libro son las herramientas de ese administrador; la Parte III, su método.

Qué aprender (y qué no), sin FOMO

Contra el miedo, muchos reaccionan con FOMO: intentar aprenderlo todo, cada herramienta nueva, cada lunes. Mal camino: te quema y no acumula.

El mapa correcto es el de este libro. Primero el modelo y sus límites (capítulo 4): todo lo demás se construye sobre entender qué es lo que piensa. Luego las piezas que convierten un agente genérico en tu equipo: reglas, memoria, protocolo de herramientas, oficios. Al final, la dirección: especificaciones, tests, revisión. El resto del ecosistema (cada app nueva, cada framework de la semana) son variaciones sobre esas mismas piezas: cuando entiendes el patrón, la novedad se estudia en una tarde.

Mi criterio para lo nuevo: entiende de qué trata antes de estudiarlo a fondo. Si una herramienta toca una pieza que ya conoces, la novedad es cosmética y la lees de corrido. Si toca una pieza que no tienes, ahí sí vale la tarde completa. El FOMO se cura con un mapa, y el mapa es corto.

Si eres estudiante

Dos noticias, una mala y una buena. La posible mala: según dónde estudies, te están enseñando con ejemplos de hace una década, y vas a sentir que la escuela va

tarde. La buena: los términos y el método de la ingeniería de software de los últimos veinte años siguen vigentes; lo que cambió es el ejecutor.

El ejemplo más claro es el test driven development, que verás en el capítulo 12: escribir el test que falla antes que el código era buena práctica cuando el junior era humano, y sigue siendo exactamente la misma práctica cuando el junior es un agente. Lo que enseña tu escuela sobre diseño de bases, arquitectura, procesos, se adapta cambiando una palabra del reparto: donde decía «desarrollador», ahora dice «agente». Estudia el método; el reparto lo haces tú.

El software no murió

Cerremos el argumento. El vibe coding commoditizó la aplicación básica, y eso produce más software, no menos trabajo técnico: cada sistema vibe-codeado que sobrevive y crece necesita, más temprano que tarde, a alguien que entienda lo que hay debajo. En mi propia cancha lo veo: cada vez más de lo que me piden es revisar y dirigir lo que ya existe, y el stack de este libro existe precisamente para que dirigir cueste menos que rehacer.

La forma del trabajo cambió: menos tecleo, más especificar, verificar y responder. El fondo no cambió: el software serio lo sostiene gente que entiende por qué funciona. Tu decisión de carrera es una sola: ser el que entiende y dirige, o ser el que le pide al modelo que adivine. El resto del libro es para el primero.

Preguntas de reflexión

1. De tu trabajo actual, ¿qué porcentaje es «la aplicación básica» que el vibe coding ya cubre, y qué porcentaje vive en la zona donde el proyecto crece? Sé honesto: esa proporción es tu urgencia real de cambiar de papel.

2. Piensa en el último problema técnico difícil que resolviste. ¿Cuánto fue mecánica (que un agente ya haría) y cuánto fue criterio (experiencia, decisiones, responsabilidad)? ¿Cómo enseñarle a un agente la primera parte para quedarte con la segunda?

3. Cuando una herramienta nueva aparece en tu radar, ¿cómo decides hoy si estudiarla o ignorarla? Disecciona tu criterio contra el de este capítulo (¿toca una pieza que ya tengo o una que me falta?).

Resumen de puntos clave

– El miedo profesional es real y su causa es precisa: el vibe coding (Karpathy, 2025) ya cubre la aplicación básica, incluso para personas sin formación técnica.

– El techo del vibe coding aparece cuando el proyecto crece: escala, optimización, integraciones legacy, decisiones a ciegas que se pagan con reescrituras.

– El profesional sigue siendo necesario por dos razones: experiencia (criterio de diseño que la IA continúa, no inventa) y responsabilidad (alguien responde por lo que el sistema toca; la herramienta no).

– El nuevo papel es administrador de agentes: delegar en agentes especializados, darles contexto, verificar y decidir; el conocimiento técnico pesa en las decisiones, no en el tecleo.

– Contra el FOMO: un mapa corto (modelo, piezas, dirección) y un criterio (entiende de qué trata antes de estudiar a fondo); para estudiantes: el método sigue vigente, cambia el ejecutor.

Glosario

– **Vibe coding**: programar pidiéndole la aplicación a la IA y aceptando el resultado sin revisarlo a fondo; término acuñado por Andrej Karpathy en 2025.

– **Administrador de agentes**: el papel de dirigir agentes especializados (delegar, dar contexto, verificar, decidir), como antes se administraba un equipo de desarrolladores.

– **FOMO** (fear of missing out): miedo a quedarse fuera de las novedades; en este contexto, la reacción de intentar aprenderlo todo sin mapa.

– **CRUD**: las cuatro operaciones básicas de una aplicación de datos (crear, leer, actualizar, borrar); ejemplo arquetípico del trabajo ya comoditizado.

Ejercicio

Haz la auditoría de tu semana laboral: lista las 10 tareas que más tiempo te consumieron y clasifica cada una en tres cajones: «ya la haría un agente bien dirigido», «requiere mi criterio (diseño, decisión, responsabilidad)» y «requiere las dos cosas». Con las del primer cajón, escribe qué pieza del stack (reglas, memoria, MCP, skills, dirección) haría falta para delegarla de verdad. Esa lista es tu plan personal de lectura de la Parte II: los capítulos que te salgan repetidos, llévalos primero.

Capítulo 2. Antecedentes: mi línea del tiempo

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Reconocer las tres etapas por las que pasa casi todo programador que adopta IA: el intermediario manual, el primer arnés y el stack dirigido.
2. Señalar con precisión qué cambió cuando el agente dejó de necesitar intermediario, y qué problemas no se arreglaron con eso.
3. Usar mi línea del tiempo como espejo para ubicar en qué punto estás tú hoy.

Octubre de 2025: el intermediario

En octubre de 2025 entré a trabajar como desarrollador, y mi stack de IA era este: ChatGPT en el navegador y Visual Studio Code. Suena primitivo escrito así; era lo normal.

El flujo de un ticket era una ceremonia. Me asignaban el issue en Jira. Buscaba el código relacionado por mi cuenta, abriendo archivos hasta dar con el lugar probable. Cuando algo fallaba, debuggeaba en la terminal, copiaba el mensaje de error, se lo pegaba a ChatGPT, copiaba lo que me devolvía y lo pegaba en el código. Corría de nuevo. Fallaba de nuevo. Repetía el ciclo: mi record personal fue resolver un ticket en el décimo intento, y no todos los intentos los cerraba la IA; algunos los terminaba resolviendo yo mismo, y otros los sacaba con Gemini web cuando ChatGPT se quedaba atrapado. Al final creaba mi rama manualmente, levantaba el pull request y esperaba la revisión del DevOps.

La descripción honesta de ese octubre: la mayoría del trabajo la hacía yo manualmente, con ayuda de la IA, sin saber exactamente qué era lo que estaba usando, si funcionaba de verdad, o si lo que hacía era efectivo u óptimo. Funcionaba porque funcionaba: el ticket cerraba, el PR pasaba, el ciclo volvía a empezar.

Míralo con el lente de hoy, porque aquí vive la lección del capítulo. En ese flujo, el modelo no veía mi código: yo le traducía pedazos, un error a la vez, y le devolvía sus parches. Yo era los ojos y las manos de la IA, y la IA era mi adivino de cabecera. La industria le pondría nombre a ese rol más adelante: intermediario humano entre el modelo y el código. Todo el contexto que yo acumulaba debugging se perdía en cada pegado; cada intento arrancaba de nuevo. Ni siquiera sabía que existía otra forma de trabajar.

Noviembre de 2025: el primer arnés

Después de resolver dos o tres tickets de la manera anterior, mi jefe nos obligó a cambiar: nos migró a Antigravity, el IDE agéntico de Google que se había

lanzado en preview gratuito el 18 de noviembre de 2025, construido alrededor de Gemini 3 (Google, 2025).

El salto se sintió de inmediato, y conviene nombrarlo con precisión: con ese arnés de IA, dejé de ser el intermediario. El agente veía mi código directamente y podía editarlo él mismo; yo dejé de copiar errores y pegar respuestas. La parte mecánica de mi ceremonia de octubre (traducir el problema, transportar el contexto, transcribir la solución) desapareció en un día.

Lo que no desapareció fueron los problemas de fondo de la IA. Sin configuración adecuada, el agente alucinaba (inventaba, con total seguridad, funciones y causas que no existían), se inventaba cosas sin verificar, y no investigaba en internet por su cuenta: contestaba con lo que traía en la panza. Un arnés mejor le había dado manos y ojos al modelo; el criterio y la verificación seguían siendo un problema mío, y yo todavía no sabía ni que había un manual para eso.

De ahí en adelante

Ese noviembre cerró mi etapa de intermediario y abrió la pregunta que se volvió este libro: si el modelo ya puede ver y editar mi código, ¿qué falta para que trabajar con él sea de verdad mejor que trabajar solo? La respuesta, aprendida a lo largo de un año, es todo lo que sigue: entender qué es el modelo y sus límites (capítulo 4), qué es exactamente ese arnés que me instalaron (capítulo 5), cómo darle reglas, memoria, herramientas y oficios (Parte II), y cómo dirigirlo para que el trabajo sea verificable (Parte III). El capítulo que sigue pone la idea que ordena todo ese camino.

Si vienes de mi misma octubre-2025: el ciclo de copiar y pegar no era tu incapacidad; era una etapa. Lo que sigue es el mapa para salir de ella.

Preguntas de reflexión

1. ¿En qué punto de esta línea del tiempo estás hoy: intermediario manual, primer arnés, o dirigiendo un stack? ¿Qué evidencia concreta de tu semana te lo dice?

2. En mi ceremonia de octubre, yo hacía de ojos y manos del modelo. En tu flujo actual con IA, ¿qué partes sigue haciendo un humano que podría hacer el agente, y qué partes no debería hacer nadie sin verificar?

3. Cuando un agente te devuelve código que funciona, ¿qué te dice tu experiencia (o tu octubre propio) sobre la diferencia entre «funcionó» y «está bien hecho»?

Resumen de puntos clave

– Octubre de 2025: desarrollo manual con IA de chat (buscar código, copiar error, pegar respuesta, décimo intento, rama y PR a mano), funcionando a ciegas sobre qué era efectivo u óptimo.

– El rol oculto de esa etapa: intermediario humano, transportando contexto en ambas direcciones y perdiéndolo en cada intento.

– Noviembre de 2025: la migración forzada a Antigravity (IDE agéntico de Google, 18 de noviembre de 2025) eliminó la intermediación: el agente ve y edita el código directamente.

– Lo que el primer arnés no arregló: alucinación, invenciones sin verificar, falta de investigación propia; el criterio siguió siendo del humano.

– El resto del libro es el camino que siguió a esa pregunta: modelo, arnés, reglas, memoria, herramientas, oficios y dirección.

Glosario

– **Intermediario humano**: la persona que transporta contexto entre el modelo y el código a mano (copiar el error, pegar la respuesta); el rol que un buen arnés elimina.

– **Arnés (harness)**: todo lo que envuelve al modelo para volverlo trabajador (herramientas, memoria, bucle); anticipado aquí por primera vez, despedazado en el capítulo 5.

– **IDE agéntico**: entorno de desarrollo (editor) construido alrededor de un agente que ve y edita el código directamente, como Antigravity.

– **Alucinación**: respuesta inventada con seguridad total; el modelo produce lo plausible, no lo verificado (capítulo 4 la explica a fondo).

Ejercicio

Dibuja tu propia línea del tiempo en tres filas, con una columna por época: hace doce meses, hoy y en seis meses. Para cada época responde tres cosas: ¿con qué herramientas trabajabas (o trabajarás)?, ¿quién era el intermediario entre el modelo y el código?, y ¿qué no podías verificar? Al terminar, marca en qué fila estás más cómodo y en cuál quieres estar; esa marca es tu razón para leer la Parte II.

Capítulo 3. La idea que lo ordena todo

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Explicar por qué elegir el modelo es la decisión menos importante de un stack de desarrollo con IA.
2. Nombrar las cuatro piezas del «alrededor» que de verdad deciden el resultado.
3. Usar este mapa para ubicar cada capítulo del libro antes de leerlo.

La confesión con la que empiezo todo

Cuando empecé a desarrollar con inteligencia artificial, creía que el asunto era elegir bien el modelo. La línea del tiempo del capítulo anterior lo cuenta en detalle: meses de copiar y pegar entre mi terminal y un chat, sin sospechar que el problema era otro. Me equivoqué, y el error me costó meses.

El modelo es el ingrediente más barato de cambiar de toda la receta. Cambiarlo es, literalmente, editar una línea de configuración: mismo proyecto, mismo agente, mismo historial, cerebro nuevo. Si mañana sale uno mejor, lo adopto en la tarde. Una decisión que se corrige con una línea no es la decisión que define tu resultado (Valverde, 2026).

Lo que de verdad cambia el resultado es todo lo que está alrededor del modelo: si el agente recuerda tu proyecto de una sesión a la siguiente, si conoce tus reglas y convenciones, si puede abrir tus tickets y leer la documentación real de tus librerías, y sobre todo, si tú diriges el proceso o nomás ves qué sale. Ese «alrededor» es el tema completo de este libro.

El alrededor tiene nombre: harness

Resulta que la conversación de la industria llegó al mismo lugar que mi dolor de meses. A Martin Fowler, una de las voces con más criterio en ingeniería de software, le tocó ponerle nombre al fenómeno en abril de 2026: «The term harness has emerged as a shorthand to mean everything in an AI agent except the model itself» (Fowler, 2026). El arnés (harness) es todo lo que envuelve al modelo para volverlo trabajador: instrucciones, herramientas, memoria, verificación.

Que el vocabulario exista cambia algo práctico: puedes mejorar tu arnés sin esperar a nadie. Los modelos salen de laboratorios que no controlas, con precios y cuotas que tampoco controlas. El arnés es tuyo. Cada capítulo de este libro es una pieza de arnés que tú puedes construir hoy, con las herramientas que ya tienes.

Fowler llama a esta disciplina harness engineering, ingeniería de arneses, y la divide en dos: lo que trae el proveedor de fabrica (su system prompt, su recuperación de código) y el arnés externo que construye el usuario para su caso:

guías que lo orientan antes de actuar y sensores que lo corrigen después (Fowler, 2026). Este libro vive casi entero en esa segunda mitad.

Las cuatro piezas del alrededor

Desglosemos el «alrededor» en las cuatro piezas que decidirán tu experiencia, tal como las vivo yo.

Pieza 1: memoria. Un chat nuevo amanece sin recordar nada. Con memoria persistente, el agente retoma tu arquitectura, tus convenciones y tus bugs ya enterrados sin que los re-expliques. Es el tema del capítulo 9.

Pieza 2: reglas. Tu forma de trabajar, escrita una vez en un archivo que cualquier agente obedece: idioma, convenciones de commit, límites, y una escalera anti-sobreingeniería que te va a ahorrar mucho código basura. Capítulo 7, y de lo más rentable del libro.

Pieza 3: contexto. Lo que el agente puede tocar: tus tickets de Jira, la documentación vigente de tus librerías (no la de hace dos años, que el modelo memorizó en su entrenamiento). El protocolo MCP, capítulo 10, es el que conecta todo esto.

Pieza 4: dirección. El cómo le encargas el trabajo. Para cambios grandes, especificaciones por fases con agentes frescos revisándose entre ellos; para cambios chicos, la solución más corta que funcione. La Parte III completa.

Si afinas la mirada, notarás el patrón: las cuatro piezas tratan de lo mismo, que el agente trabaje con TU contexto y bajo TU criterio, en lugar de improvisar con el suyo.

El trato didáctico de este libro

Cada capítulo sigue la misma estructura, porque la estructura es la mitad de la enseñanza: qué es la pieza (explicada desde cero, sin asumir que conoces nada de IA), por qué importa (qué problema te resuelve y cuánto cuesta no tenerla), cómo se usa (configuración y práctica real), y un ejemplo de mi trabajo que no inventé para la ocasión.

Los números van en el cuerpo del texto, no escondidos en notas: si algo cuesta $18 USD al mes, lo vas a leer aquí mismo. Las cifras tienen fecha, porque este mundo se mueve rápido: todo lo que afirmo sobre herramientas vale «a octubre de 2026», y lo dice el texto cuando importa.

El mapa completo

La Parte I abrió con lo que está en juego (el capítulo 1) y con mi línea del tiempo (el capítulo 2); este capítulo puso la tesis, y la cierran los fundamentos: el capítulo 4 explica qué es un modelo de lenguaje (tokens, contexto, por qué alucina) y el capítulo 5 separa modelo, agente y aplicación, tres capas que casi todos confundimos. La Parte II presenta el stack pieza por pieza: las dos apps

donde vivo el agente, el archivo de reglas, los modelos GLM y su perilla de razonamiento, la memoria, el protocolo MCP y las skills. La Parte III muestra la dirección del trabajo: especificaciones, sub-agentes, tests como árbitro, diseño con IA y los tres terrenos donde pruebo todo. El capítulo 15 condensa el mínimo viable para armar el tuyo.

Con el mapa en mano, la tesis del libro cabe en una pregunta que te conviene hacerte antes de girar la página: cuando termine la Parte III, ¿vas a ser de los que dirige el proceso, o de los que nomás ven qué sale?

Preguntas de reflexión

1. De las cuatro piezas del alrededor (memoria, reglas, contexto, dirección), ¿cuál te falta más en tu trabajo de hoy, y cómo le has dado la vuelta hasta ahora?

2. Fowler divide el arnés en el que trae el proveedor y el que construyes tú. ¿Cuáles partes de tu flujo de trabajo actual dependen de decisiones de un proveedor que no controlas?

3. Si mañana tu modelo favorito desapareciera, ¿cuánto de tu forma de trabajo sobreviviría intacta?

Resumen de puntos clave

– El modelo es el ingrediente más barato de cambiar del stack; se swappea con una línea de configuración.

– El resultado lo decide el alrededor: memoria, reglas, contexto y dirección.

– El alrededor tiene nombre en la industria: harness, «todo lo que hay en un agente de IA excepto el modelo mismo» (Fowler, abril de 2026).

– El arnés externo es tuyo: puedes mejorarlo hoy, sin esperar a ningún laboratorio.

– El libro se lee corrido o por capítulo-pieza; solo se asume que sabes programar.

Glosario

– **Stack**: el conjunto completo de herramientas y prácticas con las que trabajas; aquí, el de desarrollo con IA.

– **Harness (arnés)**: todo lo que envuelve a un modelo para convertirlo en trabajador: instrucciones, herramientas, memoria, verificación.

– **Modelo**: el cerebro que predice texto; se define a fondo en el capítulo 2.

– **Agente**: el modelo más su arnés, capaz de actuar; se define a fondo en el capítulo 3.

Ejercicio

Antes de seguir leyendo, escribe en cuatro renglones tu propio diagnóstico: una línea por pieza del alrededor (memoria, reglas, contexto, dirección), anotando qué tienes resuelto y qué improvisas en cada una. Guárdalo: el capítulo 15 te va a pedir exactamente ese diagnóstico para armarte un plan de adopción a tu medida.

Capítulo 4. ¿Qué es un modelo de lenguaje?

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Explicar con tus palabras qué hace un modelo de lenguaje y qué es un token.
2. Explicar por qué un modelo alucina y cómo se combate con fuentes vivas.
3. Usar la ventana de contexto y la multimodalidad para decidir qué le das a un modelo y cómo.
4. Ubicar cualquier modelo en el mapa de frontera, pesos abiertos y locales.

La máquina de predecir texto

Un modelo de lenguaje hace una sola cosa: recibe texto y calcula, pieza por pieza, cuál es el fragmento más probable que sigue. Eso es todo. Cuando le pides «escribe una función que valide un correo», el modelo no consulta un catálogo de funciones ni razona como una persona: genera la continuación más plausible de tu pedido, fragmento tras fragmento, hasta que la estadística dice que el texto se agotó.

Suena a poco, y produce demasiado. Entrenado con cantidades industriales de texto (código, documentación, foros, libros), ese mecanismo de predicción aprende patrones tan finos que termina componiendo programas que compilan, ensayos que argumentan y respuestas que parecen pensadas. El truco está en la escala: mismo mecanismo simple, billones de ejemplos.

Para tu vida como programador, la palabra clave de ese párrafo es «plausible». El modelo produce lo que estadísticamente encaja, y casi siempre el código que encaja también funciona. Casi siempre. Esa brecha entre plausible y verificable es la responsable de la mitad de tus frustaciones con la IA, y la vamos a cerrar con piezas del stack, no con optimismos.

Tokens: la moneda del modelo

El modelo lee tokens: fichas de texto de tamaño variable que pueden ser una palabra completa («función»), un pedazo («ing» de una palabra larga) o un símbolo suelto («{», «=>»). Antes de que el modelo piense nada, tu texto se parte en fichas; cuando responde, va soldando fichas una por una.

Las fichas importan porque todo se cobra en ellas. El ejemplo con código real: la línea `const precio = obtenerPrecioBase(carrito) * 1.16;` ocupa alrededor de 15 tokens, y una función típica de 20 líneas, entre 250 y 400. Un archivo de 500 líneas ya ronda los 8,000 tokens. Cuando en el capítulo 8 hablemos de cuotas y precios, la moneda va a ser esta; conviene que desde ahora la tengas fichada.

Nadie cuenta tokens a mano, ni hace falta: cualquier herramienta decente te muestra el consumo por sesión. Lo que sí necesitas es la intuición de escala, porque de ella salen decisiones reales: si un archivo cabe completo en tu contexto, se lo das completo; si no, decides qué pedazo entra.

Por qué alucina (y por qué no se va a «arreglar»)

Un modelo alucina cuando predice con fluidez algo falso: una función de una librería que no existe, un parámetro inventado, una fecha con seguridad de noticiero. La alucinación sale del mecanismo mismo, y por eso acompaña a todo modelo por bueno que sea: el modelo produce la continuación más plausible de tu texto, y la continuación más plausible de «la función para X en la librería Y es…» puede ser una función real o una muy verosímil que nadie escribió jamás (Valverde, 2026).

El ejemplo clásico de este mundo: la IA te sugiere una API que fue removida hace dos años. Le pasó a todo mundo. El modelo la memorizó en su entrenamiento, tu pedido encaja con ella, y la suelta con la misma seguridad que una correcta. No sabe que está desactualizada, porque «saber» no es lo que hace.

De aquí sale una regla que el libro va a repetir hasta el cansancio: nunca confíes sin verificar, y verifica con fuentes vivas. La pieza del stack que lo resuelve se llama Context7 (capítulo 10): documentación vigente, inyectada al contexto justo cuando el agente la necesita. El clásico desastre de la API fantasma desapareció de mi flujo el día que la conecté.

La ventana de contexto: tu mesa de trabajo

La ventana de contexto es cuánto texto le cabe al modelo en una sola llamada: todo lo que le des (tus instrucciones, el código, la plática previa) debe caber en esa mesa. Lo que no cabe, no existe para el modelo en ese momento.

Los tamaños actuales van de lo cotidiano a lo escandaloso. Un contexto de 200 mil tokens (200K) ya es pan de cada día, y los modelos insignia de GLM llegan a un millón (1M): un proyecto mediano completo cabe de un bocado (Valverde, 2026). Para qué sirve en la práctica: explorar un codebase desconocido sin trocearlo, pedirle al agente que cruce un archivo contra otro que no le habías dado, o mantener una sesión larga sin que se le caiga el inicio.

Dos advertencias de usuario experimentado. Primera: la mesa grande no limpia sola; llenar el contexto de basura degrada la calidad de las respuestas, y un agente que ya leyó cincuenta archivos deja de ver el bosque (esto reaparece en el capítulo 12, cuando hablemos de sub-agentes con contexto fresco). Segunda: más contexto cuesta más cuota, así que en el capítulo 8 vas a ver que a veces conviene el modelo de ventana chica para lo mecánico.

Modelos que ven: la multimodalidad

Un modelo multimodal procesa más que texto: imágenes, y en algunos casos audio. Para el trabajo con agentes, la versión útil de hoy es «el modelo ve capturas de pantalla». Le mandas el screenshot de tu app con el botón descuadrado y te dice qué CSS arregla, sin que traduzcas el problema a palabras.

En mi stack, GLM-5.3-Flash es la variante que ve imágenes, y la uso más de lo que esperaba: revisar un diseño contra el prototipo, leer un diagrama de arquitectura, descifrar un error que solo aparece en una foto de la pantalla de alguien (Z.ai, 2026b). Es el tipo de capacidad que no sabías que necesitabas hasta que la tienes a la mano.

Frontera, abiertos y locales: el mapa de modelos

Con lo anterior ya puedes leer cualquier noticia de modelos sin perderte, porque todas caen en tres territorios del mismo mapa.

Los modelos de frontera (frontier) son la cima de capacidad: los entrenan los grandes laboratorios con cantidades de cómputo que solo ellos tienen, y los consumes en la nube, por suscripción o por uso. Los modelos de pesos abiertos (open weights) publican sus pesos, los parámetros entrenados que hacen al modelo ser lo que es, y cualquiera puede descargarlos e incluso correrlos. Y los modelos locales son los abiertos corriendo en tu propia máquina, con herramientas como Ollama (capítulo 15): tu GPU, tu privacidad, cero costo por token.

La sorpresa de 2026 es que la frontera y lo abierto ya no son mundos separados. El GLM-5.3 con el que trabajo es de pesos abiertos y a la vez compite en la frontera: Z.ai lo presenta como su modelo insignia y como «el modelo de pesos abiertos más capaz para código» a la vez (Z.ai, 2026c). Por eso la distinción que de verdad decide tu cartera no es abiertos contra cerrados, sino cómo lo consumes: la suscripción de nube (desde $18 USD al mes, capítulo 8) te da la cima sin hardware; lo local te da privacidad total a cambio de ser tú quien compre, configure y mantenga la potencia.

Mi posición, fechada a octubre de 2026: para trabajo agéntico sostenido, la nube gana por goleada; lo local brilla cuando la privacidad manda o no hay conexión. El capítulo 15 vuelve a este mapa cuando toque la frontera honesta del libro.

Sin memoria entre llamadas

Aquí va el dato estructural del capítulo, el que sostiene medio libro: un modelo es stateless, sin estado. Entre una llamada y la siguiente no recuerda nada por sí mismo. El glosario de agentes de Hugging Face lo dice plano: los modelos «have no memory between calls» (Hugging Face, 2026). Si en tu chat el modelo «recuerda» lo que dijiste hace un rato, es porque la aplicación le está reenviando la conversación completa en cada llamada, fichas mediante.

Dale una vuelta más, porque explica tres capítulos futuros de una vez. ¿Por qué el agente olvida tu proyecto al cerrar la sesión? Porque el modelo no tiene dónde guardarlo. ¿Para qué sirve la memoria persistente del capítulo 9? Para que ese «recuerdo» viva fuera del modelo y se le inyecte cuando hace falta. ¿Por qué importa tanto administrar el contexto? Porque el contexto es la única memoria que el modelo tiene en el momento de trabajar. Anthropic le llama a esto aumentar al LLM: darle recuperación, herramientas y memoria por fuera (Anthropic, 2024a). Tu stack es, en buena parte, ese «por fuera».

Tu palanca real

Cerramos con la consecuencia práctica. Si el modelo sabe lo que sabe por su entrenamiento, y no lo puedes reentrenar desde tu escritorio, entonces la única palanca que controlas es qué le llega en el contexto: qué archivos ve, qué documentación fresca recibe, qué reglas tuyas lleva puestas, qué memoria le inyectas. Dos programadores con el mismo modelo y contextos distintos obtienen resultados de otra liga.

Esa es la tesis del capítulo 1 dicha en mecánica de modelos, y de aquí en adelante el libro es un catálogo de formas de llenar bien esa mesa.

Preguntas de reflexión

1. Piensa en la última vez que una IA te inventó algo con seguridad (una API, un dato, un comando). ¿Qué había en tu pedido que hizo plausible esa invención en particular?

2. Tu proyecto actual: ¿cabe en una ventana de 200K tokens, de 1M, o en ninguna? ¿Cómo lo trocearías para que un agente lo explore bien?

3. De todo lo que le das hoy a un modelo (instrucciones, código, conversación), ¿qué proporción es basura que solo ocupa la mesa?

Resumen de puntos clave

– Un modelo de lenguaje predice el siguiente token; todo lo que «sabe» viene de entrenamiento estadístico masivo, y produce lo plausible, no lo verificado.

– El token es la moneda de todo: consumo, precios y contextos se miden en fichas (una función de 20 líneas ≈ 250 a 400 tokens).

– Alucinar es consecuencia del mecanismo, no un bug: se combate con fuentes vivas de documentación, no con buena fe.

– La ventana de contexto es tu mesa de trabajo: 200K cotidianos, 1M en las insignias; mesa más grande no es mesa más limpia.

– El modelo no tiene memoria entre llamadas: todo «recuerdo» es contexto que alguien (tu stack) le inyecta.

– El mapa de modelos tiene tres territorios: frontera (cima, por suscripción), pesos abiertos (descargables; los GLM compiten ahí arriba) y locales (en tu máquina: privacidad a cambio de hardware).

Glosario

– **Token**: ficha de texto (palabra, pedazo o símbolo) en la que el modelo lee y escribe; la unidad de medida de consumos y contextos.

– **Ventana de contexto**: cantidad máxima de tokens que caben en una llamada; todo lo que no cabe, el modelo no lo ve.

– **Alucinación**: respuesta fluida y verosímil pero falsa, producto de predecir lo plausible en lugar de consultar lo real.

– **Multimodal**: modelo que procesa más de texto; en la práctica de agentes, ver imágenes y capturas de pantalla.

– **Stateless (sin estado)**: que no conserva información entre llamadas; propiedad estructural del modelo.

– **Modelo de frontera (frontier)**: el de mayor capacidad de su momento, entrenado por un gran laboratorio y consumido en la nube.

– **Pesos abiertos (open weights)**: modelo cuyos parámetros entrenados se publican y cualquiera puede descargar y correr.

– **Modelo local**: modelo de pesos abiertos corriendo en tu propia máquina (típicamente con Ollama).

Ejercicio

Toma un archivo real de tu proyecto (uno que conozcas bien) y estímalo en tokens sin herramientas: cuenta líneas, multiplica por 12 a 20 tokens por línea de código, y anótalo. Después repite con todo tu directorio de código. Al final responde: ¿tu proyecto cabe completo en 200K tokens, en 1M, o ninguno de los dos? Esa respuesta es la primera decisión de tu futuro stack: si te conviene un modelo de contexto gigante para explorarlo, o trocearlo con estrategia.

Capítulo 5. ¿Qué es un agente? ¿Y una app?

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Distinguir modelo, agente y aplicación, y decir qué capa resuelve qué problema.
2. Describir el bucle de un agente (observar, decidir, actuar, verificar) con un ejemplo de código real.
3. Ubicar los fallos comunes de las herramientas de IA en la capa correcta, para no culpar al motor por fallas del conductor.

Las tres capas que todos confundimos

Hay tres palabras andando sueltas en toda conversación sobre desarrollo con IA: modelo, agente y aplicación. Casi todos las mezclamos (yo incluido, y por eso existe este capítulo): decimos «el agente ZCode» o «el modelo me editó el archivo», y cada mezcla esconde un malentendido que después cuesta calibrar expectativas y diagnosticar problemas.

La buena noticia: el despiece cabe en una analogía de tres piezas que ya traes de la vida real. El modelo es el motor. El agente es el conductor completo. La aplicación es el auto. Vamos capa por capa, porque cada una la vas a comprar, configurar y regañar por separado.

Capa 1: el modelo, el motor

El modelo es el cerebro que predice texto, tal como lo armamos en el capítulo 2: stateless, sin manos, capaz de razonar sobre código pero incapaz de abrir un archivo por sí mismo. Piensa, y ahí se acaba su repertorio.

Como pieza de compra, el motor es el que menos te debe preocupar, por la tesis del capítulo 1: se swappea con una línea de configuración. Los capítulos de modelos (el 8) hablan de cuál usar y cuándo, pero ya sabes que ahí no está la partida.

Capa 2: el agente, el conductor completo

Un agente es el modelo más su arnés: el software que convierte a ese cerebro en trabajador. La fórmula la resume Martin Fowler en una línea: «Agent = Model + Harness» (Fowler, 2026). En español directo: agente igual a modelo más todo lo demás.

¿Qué trae ese «todo lo demás»? Cuatro cosas, y conviene memorizarlas porque son el índice de la Parte II de este libro. Objetivos: instrucciones que le dicen qué perseguir y con qué criterio. Herramientas: manos concretas para leer y editar archivos, correr comandos, buscar en la web. Memoria y contexto: lo que sabe del

proyecto mientras trabaja. Y el bucle: el mecanismo que encadena pasos hasta acabar la tarea, que merece sección propia.

El bucle: observar, decidir, actuar, verificar

Así trabaja un agente de verdad, en ciclos. Observa: recibe tu pedido y lo que ve del proyecto (archivos, errores, memoria). Decide: elige el siguiente paso concreto (abrir un archivo, correr un test, preguntarte algo). Actúa: ejecuta ese paso con sus herramientas. Verifica: mira qué salió, compara contra el objetivo, y si falta algo, vuelve a observar. Repite hasta que la tarea cierra o se atora y te avisa.

Con un pedido real se ve así. Le digo: «el test de carrito está fallando en CI, arréglalo». Observa: corre la suite y lee el error. Decide: la hipótesis es el mock del servicio de precios. Actúa: abre el archivo del mock y corrige el caso que faltaba. Verifica: corre el test otra vez; sigue en rojo; observa el nuevo error, descarta su hipótesis y prueba la siguiente. Tres ciclos después, verde, y me muestra el diff. Sin bucle, esto sería una conversación de copiar y pegar; con bucle, es un trabajador al que le encargas y te trae resultado.

Este bucle es también el que separa dos palabras que la industria usa con precisión desde el ensayo «Building Effective Agents» de Anthropic: un workflow sigue pasos fijos que tú definiste; un agente dirige sus propios pasos dentro de tu pedido (Anthropic, 2024a). Lo que este libro llama agente es lo segundo: autonomía de método, bajo tu dirección de objetivo.

Capa 3: la aplicación, el auto

La aplicación es el entorno donde vives al agente: el producto que instalas y abres cada día. Terminal (como el CLI de ZCode o el OpenCode de interfaz de texto), aplicación de escritorio (ZCode, el Desktop de OpenCode), editor (una extensión dentro de tu IDE) o navegador. La app aporta lo que el agente no tiene: interfaz para ti, gestión de sesiones, permisos y botones, mercado de plugins, calendarización de tareas.

Un detalle que deshace mucha confusión: un mismo agente puede vivir en varias superficies. ZCode es una sola casa con tres puertas (escritorio, web y terminal), el mismo motor de agente en cada una (Valverde, 2026). Cuando cambias de app, cambia tu experiencia de conductor; cuando cambias de modelo, cambia el motor. Las dos perillas son independientes, y este libro las va a girar por separado.

Con las tres capas en la mesa, el error de dicción del principio se ve completo: decir «el agente ZCode» mezcla la capa 3 con la 2. ZCode es la aplicación; el agente es lo que corre adentro. Parece pedantería de glosario hasta que el vocabulario te cobra: si «el agente» va lento, ¿qué va lento, el motor (modelo saturado), el conductor (arnés mal configurado) o el auto (app con problemas de sincronización)?

Por qué ubicar la capa importa: los bugs por capa

Cada familia de fallos vive en su capa, y culpar a la capa equivocada te hace perder tardes.

Fallas de modelo: alucinaciones, razonamiento flojo en problemas duros, respuestas desactualizadas. Se arreglan cambiando de motor o dándole mejor contexto, nunca reinstalando la app. Fallas de agente: contexto inflado que degrada decisiones, permisos mal configurados, herramientas que no hacen lo que el modelo cree. Se arreglan en la configuración del arnés (reglas, memoria, herramientas: capítulos 7 al 11). Fallas de app: sesiones que no sincronizan, plugins que no arrancan, instaladores rotos. Se arreglan actualizando, reportando o cambiando de aplicación.

El ejemplo reciente y doloroso: en septiembre de 2026, ZCode subía silenciosamente proyectos completos a la nube. Culpar «al modelo» habría sido absurdo: el modelo jamás tocó esos archivos. Era una falla de aplicación, de las que se arreglan con código abierto y auditorías (la historia completa, con sus cifras, en el capítulo 4). Diagnosticar por capa es la diferencia entre parchar y desinstalar con enojo.

Puentes hacia el resto del libro

Ya tienes el vocabulario estructural: motor, conductor, auto, y el bucle que hace del conductor un trabajador. La Parte II presenta los concretos: en el capítulo 6 verás las dos aplicaciones donde vivo mi agente y las dos filosofías que representan. Cada pieza que sigue (reglas, modelos, memoria, protocolo, skills) es una mejora a una parte específica del conductor. Y cuando llegue la Parte III, el bucle que aprendiste aquí será la unidad mínima de la orquestación: agentes frescos ciclando por fases.

Preguntas de reflexión

1. La última frustración que tuviste con una herramienta de IA, ¿en qué capa estaba: motor, conductor o auto? ¿Cómo lo sabes?

2. El bucle observar-decidir-actuar-verificar describe también cómo trabajas tú cuando te encargan un bug. ¿En qué paso te ayuda más delegar a un agente, y en cuál te duele más?

3. Fowler: «Agent = Model + Harness». De las cuatro partes del arnés (objetivos, herramientas, memoria, bucle), ¿cuál está hoy más débil en tu flujo de trabajo?

Resumen de puntos clave

– Tres capas: modelo (motor, piensa), agente (modelo más arnés: objetivos, herramientas, memoria y bucle), aplicación (el entorno donde vives al agente).

– El bucle observar → decidir → actuar → verificar es lo que convierte predicción de texto en trabajo encargable.

– Workflow sigue pasos fijos; agente dirige sus pasos. Este libro trata de lo segundo.

– Un mismo agente puede vivir en varias superficies; cambiar de app cambia la experiencia, cambiar de modelo cambia el motor.

– Cada familia de bugs vive en una capa; diagnosticarla bien ahorra tardes (y desinstalaciones inútiles).

Glosario

– **Agente**: el modelo más su arnés: objetivos, herramientas, memoria y el bucle que encadena pasos hasta cerrar la tarea.

– **Arnés (harness)**: todo lo que envuelve al modelo para volverlo trabajador; se mejora sin esperar al proveedor.

– **Aplicación (app)**: el entorno donde vives al agente: terminal, escritorio, editor o navegador.

– **Bucle**: ciclo observar → decidir → actuar → verificar, repetido hasta cerrar la tarea.

– **Workflow**: secuencia de pasos fijos definida por ti; distinta de un agente, que dirige sus propios pasos.

Ejercicio

Toma la última tarea que le encargaste a una IA (o inventa una razonable) y escríbela como bucle: para cada ciclo, una línea de qué debería observar el agente, qué decidir, qué actuar y con qué la verificaría. Hazlo hasta tres ciclos. Si te atoras en «actuar», te falta herramienta; si te atoras en «verificar», te falta criterio: ambas respuestas te dicen qué pieza del stack te urge más de la Parte II.

Parte II. El stack

Capítulo 6. ZCode y OpenCode: dos filosofías

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Describir qué es cada herramienta (ZCode y OpenCode), a qué capa del modelo de tres capas pertenece, y qué filosofía representa cada una.
2. Compararlas con criterio propio usando la tabla honesta: madurez, para quién, y qué vigilar de cada una.
3. Convertir el incidente de privacidad de ZCode (septiembre de 2026) en una práctica permanente: preguntar qué se envía, a dónde y con qué llave.

Dos filosofías de la misma pregunta

La pregunta es una sola: ¿dónde va a vivir tu agente? El capítulo 5 te dio el vocabulario (el agente corre en un arnés, y la app es el auto donde lo vives); este capítulo presenta los dos autos en los que yo conduzco todos los días, y conviene decirlo desde el inicio: representan filosofías opuestas de la misma industria.

ZCode apuesta por la casa integrada: una aplicación hecha por el mismo laboratorio que hace los modelos, donde todo funciona con todo y las funciones nuevas llegan primero. OpenCode apuesta por la casa neutral: un agente que no pertenece a ningún laboratorio y te deja ponerle el cerebro que quieras. Ninguna de las dos es «la correcta»; elegir entre ellas es decidir qué quieres que sea fácil en tu día a día.

ZCode: la casa del modelo

ZCode es la aplicación de escritorio de Z.ai (el laboratorio detrás de los modelos GLM): gratis, para Windows, macOS y Linux, lanzada el 2 de julio de 2026 como el entorno oficial de sus modelos, y publicada como código abierto bajo Apache 2.0 el 20 y 21 de septiembre de 2026, junto con su CLI y su interfaz web (Z.ai, 2026e). Con el vocabulario del capítulo 5: es la capa 3 completa (el auto), construida alrededor del arnés oficial de GLM.

No es un editor con un chat pegado. Le das una tarea y el agente planea, edita archivos, ejecuta comandos y verifica lo que hizo. Lo que más uso: el Goal Mode, donde le das un objetivo y arma ciclos de planear, ejecutar y verificar por su cuenta; los sub-agentes, que reparten el trabajo entre especialistas con contexto fresco cada uno; y las conexiones MCP, donde viven mis integraciones de Jira, memoria y diseño (Valverde, 2026).

Su ventaja estructural es ser el harness oficial: las funciones nuevas de los modelos GLM llegan aquí primero, sin configurar nada. Si tu stack vive en la familia GLM (el mío sí, como verás en el capítulo 8), esa ventaja se siente todos los días.

Sus desventajas también hay que decirlas: el ecosistema es joven (mercado de plugins pequeño, funciones en beta con filos), y en septiembre de 2026 tuvo un incidente de privacidad serio que merece sección propia más abajo, porque de ahí salió una de las lecciones de este libro.

OpenCode: el agente sin casa de modelos

OpenCode es la filosofía opuesta: un agente open source (licencia MIT, de la empresa Anomaly) que no pertenece a ningún laboratorio. Conectas más de 75 proveedores de modelos con tu propia llave, GLM incluido, y es el proyecto de agentes más grande del mundo abierto: más de 200,000 estrellas en GitHub a octubre de 2026 (Anomaly, 2026).

Tiene dos superficies que son el mismo motor con distinta ropa. La CLI/TUI es la original y la madura: vive en tu terminal, se puede guionizar, corre por SSH en un servidor remoto, y su cadencia de releases es casi diaria; es mi herramienta de trabajo pesado. El Desktop es el mismo motor con interfaz gráfica, cómodo para ver sesiones de un vistazo, pero es beta de verdad: las sesiones no siempre sincronizan con la CLI, algunos plugins no arrancan en la versión gráfica, y a inicio de año tuvo dos fallas de seguridad ya parcheadas (Valverde, 2026). Motor excelente, envoltorio joven.

La tabla honesta

	ZCode (desktop)	OpenCode CLI	OpenCode Desktop
Qué es	El entorno agéntico de Z.ai, open source	El motor open source en tu terminal	El mismo motor con GUI
Madurez	Joven, con casa detrás (cadencia semanal)	El más maduro del ecosistema abierto	Beta desde diciembre de 2025
Ideal para	Trabajar con GLM sin fricción, orquestación incluida	Automatizar, SSH, scripting, control total	Seguir sesiones a la vista
Ojo con	Incidente de privacidad reciente (remediado)	Curva de terminal, sin GUI	Sync de sesiones y plugins aún verdes

Una fila merece lectura adicional: la del incidente. Ahí va.

El incidente: qué pasó de verdad

En septiembre de 2026, un desarrollador que investigaba por qué su disco se llenaba descubrió que ZCode empacaba el proyecto completo del usuario (incluida toda la historia de Git) en archivos cifrados y los subía a la nube sin preguntar. La función venía activada por defecto, era invisible para el sistema de

permisos del agente, y los interruptores de la interfaz no la detenían (InfoWorld, 2026).

Las cifras documentadas del caso más grande: un archivo de 313 MB con 42,411 piezas, de los cuales el 86.6% era historia de Git: commits viejos, ramas borradas y secretos que ya nadie tenía en su copia de trabajo pero que seguían vivos en la historia. Y el cifrado era de llave ajena: los archivos iban cifrados de forma que solo el servidor de Z.ai podía abrirlos, así que el usuario no podía ni verificar qué se había llevado. En un solo caso documentado se llegaron a disparar hasta 62 snapshots por sesión (InfoWorld, 2026).

La respuesta de la empresa fue rápida y en la dirección correcta: disculpa pública, la función eliminada en la versión 3.14.0, el depósito de nube borrado completo, la app abierta bajo Apache 2.0 para que cualquiera pueda auditarla, y dos auditorías externas encargadas (Z.ai, 2026e). Y aun así, la parte incómoda: el borrado es palabra de la empresa, porque la llave de descifrado solo la tenía ella. Ejemplar y suficiente no son lo mismo.

La lección que se queda

De este incidente salió la pregunta que le repito a cualquiera que empiece con estas herramientas: ¿qué se envía, a dónde, y con qué llave? (Valverde, 2026).

Ponla a trabajar en tu propio stack hoy mismo. ¿Qué se envía: prompts solamente, o archivos completos, o historia de Git? ¿A dónde: a qué servidores, en qué país, bajo qué jurisdicción? ¿Con qué llave: cifrado que tú puedas abrir, o cifrado que solo abre el proveedor? El incidente de ZCode no es una razón para huir de ZCode (yo sigo usándolo; el código abierto permite verificar que la ruta de subida ya no existe); es una razón para hacerle esa pregunta a cada herramienta que toque tu código, sea de la empresa que sea.

Cuál elegir

Uso las dos, y no es diplomacia: la CLI de OpenCode para automatización y trabajo por SSH, ZCode para el día a día con GLM y la orquestación. Si empiezas de cero, mi consejo en una línea: elige la que esté más cerca de donde ya trabajas (terminal o escritorio), dale dos semanas de uso real, y no cambies hasta que puedas nombrar en una frase qué te falta. La pieza que de verdad iguala a cualquier app que elijas es la del siguiente capítulo: tu archivo de reglas.

Preguntas de reflexión

1. Con la tabla honesta enfrente: ¿en qué casilla cae tu necesidad de hoy, y qué casilla de «ojo con» estás dispuesto a habitar mientras madura?
2. Aplica la pregunta de las tres partes (¿qué, a dónde, con qué llave?) a la herramienta de IA que usas hoy. ¿Cuántas de las tres respuestas sabes sin consultar la documentación?

3. El incidente se remedió abriendo el código. ¿Qué puede auditar la comunidad en un repo abierto que no puede auditar en uno cerrado, y qué sigue siendo palabra del proveedor?

Resumen de puntos clave

– Dos filosofías: ZCode (la casa integrada del modelo, funciones GLM primero) y OpenCode (agente neutral, 75+ proveedores con tu llave, 200,000+ estrellas).

– OpenCode tiene dos superficies con el mismo motor: CLI madura para trabajo pesado, Desktop beta para ver sesiones.

– El incidente de septiembre de 2026: subida silenciosa de proyectos cifrados (313 MB y 86.6% de historia Git en el caso mayor), remediada con eliminación de la función, borrado del depósito, código abierto y auditorías.

– La lección permanente: qué se envía, a dónde y con qué llave; hazle esa pregunta a cualquier herramienta que toque tu código.

– Elegir app es decidir dónde vives, no cuál es mejor; la pieza que te iguala con cualquiera es tu archivo de reglas.

Glosario

– **Goal Mode**: modo de ZCode donde el agente arma ciclos de planear → ejecutar → verificar por su cuenta hasta llegar al objetivo.

– **CLI/TUI**: interfaz de línea de comandos (terminal); la superficie original y madura de OpenCode.

– **BYOK** (bring your own key): llevar tus propias llaves de API de los proveedores de modelos, en vez de pagar el paquete de una sola casa.

– **Snapshot**: copia empaquetada del estado de tu proyecto; la que causó el incidente viajaba cifrada con llave que solo tenía el proveedor.

Ejercicio

Auditoría exprés de tu herramienta actual (la que uses hoy con IA, sea cual sea): abre su documentación o configuración y busca las tres respuestas (qué se envía, a dónde, con qué llave). Anótalas en tres renglones. Si alguna respuesta no existe en la documentación, eso también es un dato: márcalo con un signo de pregunta y guárdalo; el capítulo 7 te dará la herramienta para volver a esta lista con tu agente trabajando para ti.

Capítulo 7. El archivo de reglas (`AGENTS.md`)

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Explicar qué es `AGENTS.md`, por qué se volvió el estándar de facto y dónde vive en tu máquina.
2. Redactar la primera versión de tu propio archivo de reglas con los cuatro bloques que de verdad pagan la renta.
3. Aplicar la escalera anti-sobreingeniería como criterio de decisión antes de escribir cualquier código con un agente.

El impuesto de re-explicar tu forma de trabajar

Sin reglas, cada agente llega con su propio criterio: escribe los comentarios en inglés aunque tú hables español, hace commits a su manera, te propone instalar tres librerías para resolver algo que la librería estándar ya resuelve, y en cada sesión nueva empiezas de nuevo a corregirlo. Es un impuesto pequeño por sesión que, cobrado todos los días, se vuelve carísimo (Valverde, 2026).

El arreglo es de una simpleza que casi da pena: escribir tus reglas una sola vez, en un archivo que el agente lee antes de trabajar.

Qué es `AGENTS.md` y de dónde salió

`AGENTS.md` es un archivo de texto en markdown con instrucciones permanentes para agentes de código: convenciones, preferencias, límites. La propuesta nació con OpenAI en septiembre de 2025 y se extendió tan rápido que hoy es el estándar de facto: ZCode, OpenCode, Claude Code, Codex y casi cualquier herramienta del ecosistema lo lee (agents.md, 2025). Tú escribes una vez, y aplica para siempre, en cualquier app.

Vive en dos niveles, y conviene usar los dos. El global (en tu carpeta de usuario) lleva lo que vale para todo lo que tocas: idioma, estilo, límites de tu máquina. El de proyecto (en la raíz del repositorio) lleva lo particular de ese código: convenciones del equipo, comandos de prueba, decisiones de arquitectura. El agente lee ambos y los suma.

Las cuatro cosas que de verdad pagan

Mi archivo global, destilado a lo que ha demostrado valer (Valverde, 2026):

1. Idiomas, separados por destino. En qué idioma converso con el agente y en qué idioma se escriben el código, los comentarios y los mensajes de commit. Parece menor; es lo primero que un agente sin reglas hace distinto a ti.

2. Convenciones de trabajo. Formato de los commits, herramientas de terminal preferidas, y cómo quiero que se comuniquen los cambios (cortos, sin ensayo).

3. La escalera anti-sobreingeniería. El criterio de decisión antes de escribir código; sección propia más abajo, porque es el corazón del archivo.

4. Los límites de mi máquina. La línea que más me ha salvado: mi laptop tiene 16 GB de RAM y se congela si los tests corren con paralelismo completo, así que el archivo dice que los corra limitados a 2 workers. La anécdota completa está en el capítulo 9 (cómo esa regla llegó a memoria permanente); aquí lo importante es el patrón: lo que tu cuerpo aprendió del hardware, el archivo lo hereda.

La escalera anti-sobreingeniería

Esta es la pieza del archivo que más cambio provoca en cómo trabaja tu agente. Antes de escribir una sola línea, el agente debe subir la escalera completa y solo escribir código si todos los peldaños fallaron (Valverde, 2026):

1. ¿Esto necesita existir? (si la respuesta es no, se acabó: la mejor línea de código es la que nunca se escribe)
2. ¿Ya existe en el codebase?
3. ¿Lo resuelve la librería estándar?
4. ¿Lo resuelve una feature nativa de la plataforma?
5. ¿Lo resuelve una dependencia que ya tengo instalada?
6. ¿Cabe en una línea?
7. Solo entonces: escribir código, el mínimo que funcione.

La escalera importa más con agentes que con personas, por una razón mecánica: el agente quiere complacerte. Le pides un «sistema de cache» y te construye una fábrica de caches con interfaz configurable, porque para eso le pagaste tokens. La escalera lo frena en el peldaño uno: ¿esto necesita existir? En mi experiencia, la mitad de las sobre-órdenes que le doy mueren en ese primer peldaño, y el código que no se escribió es el que nunca hay que mantener.

Una tarde bien invertida

Del capítulo 1 viene la promesa de orden de retorno; aquí se cumple la primera. Si de todo este libro solo adoptas una cosa, que sea este archivo. No necesitas la versión perfecta: abre un archivo, ponle los cuatro bloques de arriba con tus respuestas, y dale una semana de uso; lo que el agente siga haciendo mal, agrégalo como regla. La primera versión mía salió en una tarde; la actual lleva un año de pequeñas correcciones y es, con mucho, la mejor relación esfuerzo-beneficio de todo mi stack (Valverde, 2026).

Preguntas de reflexión

1. Piensa en las últimas tres correcciones que le hiciste al trabajo de una IA. ¿Cuántas eran reglas disfrazadas de corrección (idioma, formato, estilo)? Esa es tu primera tanda de reglas esperando ser escritas.

2. La escalera mata features en el peldaño uno. ¿Qué «sistema» de tu código actual no habría pasado ese peldaño, y qué te costó escribirlo y mantenerlo?

3. ¿Qué límite de tu hardware o de tu flujo (RAM, red, tiempo) merecería una línea en tu archivo para que ningún agente lo vuelva a descubrir por las malas?

Resumen de puntos clave

– `AGENTS.md`: instrucciones permanentes que casi todas las herramientas leen; propuesto por OpenAI en septiembre de 2025, estándar de facto desde entonces.

– Dos niveles: global (todo tu trabajo) y por proyecto (ese repositorio); el agente suma ambos.

– Los cuatro bloques que pagan: idiomas por destino, convenciones de trabajo, la escalera anti-sobreingeniería y los límites de tu máquina.

– La escalera (¿necesita existir? → ¿ya existe? → ¿stdlib? → ¿nativo? → ¿ya instalado? → ¿una línea? → mínimo código) importa más con agentes porque el agente quiere complacerte.

– Es la inversión de mejor retorno del stack: una tarde de trabajo, correcciones de por vida.

Glosario

– **`AGENTS.md`**: archivo de instrucciones permanente para agentes de código; estándar de facto del ecosistema.

– **Escalera anti-sobreingeniería**: secuencia de siete peldaños que el agente debe agotar antes de escribir código nuevo.

– **Worker (de tests)**: proceso paralelo que ejecuta pruebas; en máquinas justas de RAM, limitarlos evita congelamientos.

– **Sobreingeniería**: construir más de lo que el problema pide; con agentes, el riesgo sube porque complacen por diseño.

Ejercicio

Escribe la versión 1 de tu archivo de reglas ahora, no mañana: abre `AGENTS.md` en tu carpeta de usuario y llena los cuatro bloques con una línea cada uno (idiomas; commits y estilo; copia la escalera tal cual; tu límite de hardware más doloroso). Úsalo esta semana y anota cada corrección que le hagas a un agente: cada una es una regla que te falta. Al final de la semana tendrás tu versión 1.1, escrita por la realidad y no por la teoría.

Capítulo 8. Los modelos GLM y cuánto dejarlos pensar

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Describir el abanico actual de modelos GLM y qué papel juega cada uno en una semana de trabajo real.
2. Aplicar el patrón de un modelo por fase (explorar, implementar, mecanizar) en lugar de perseguir «el mejor modelo».
3. Explicar desde cero qué significa que un modelo razona y qué ajusta la perilla de esfuerzo (High/Max), para decidir cuándo pagarla.

Cómo se paga todo esto

Antes del abanico, la billetera. Uso los modelos GLM de Z.ai con su plan de programación, el GLM Coding Plan: desde $18 USD al mes, y la misma suscripción funciona dentro de más de 20 herramientas (ZCode, OpenCode, Claude Code, Codex, Cline y compañía), así que no estás casando tu dinero con una sola app (Z.ai, 2026a).

El plan se cobra en créditos con dos ventanas: una que se recarga cada 5 horas y otra cada semana. Detalle amable: fuera de hora pico el consumo cuenta a la mitad de precio. Detalle honesto: los modelos insignia queman créditos rápido en sesiones agénticas largas, y es normal ajustar el plan a tu ritmo real en el primer mes (Z.ai, 2026a).

El abanico, con fecha de caducidad

Lo que sigue describe el abanico vigente a octubre de 2026. Las versiones rotan rápido en este mundo: memorizar la tabla sirve menos que entender el patrón de uso, así que léela con el patrón en mente.

Modelo	Qué es
GLM-5.3	La insignia (agosto de 2026): un millón de tokens de contexto (1M) y razonamiento siempre encendido; no se puede apagar
GLM-5.3-Flash	La rápida y multimodal (ve capturas de pantalla): consume cerca de 3 veces menos cuota; mi default de sesión
GLM-5.2	1M de contexto y esfuerzo de razonamiento ajustable; el caballo de carga del año
GLM-5.1 / GLM-5-Turbo	La generación anterior (~200K de contexto) y la variante barata para tareas mecánicas

La insignia se presentó en agosto de 2026 como el modelo de pesos abiertos más capaz para código, con una mejora de 50% sobre su antecesor en el banco

interno de Z.ai (Z.ai, 2026c); la Flash llegó dos semanas después como la primera multimodal de la familia, con 320 mil millones de parámetros totales y 18 activos por token, el truco que la hace barata de correr (Z.ai, 2026b).

El patrón: un modelo por fase

El uso que más valor da no es elegir un modelo, es repartir la inteligencia según la fase del trabajo (Valverde, 2026).

Para explorar un codebase grande, el de contexto gigante: con 1M de tokens el agente se lee el proyecto de un bocado y las respuestas salen con el mapa completo en la cabeza. Para implementar el día a día, el default balanceado: rápido, multimodal, y con la cuota para aguantar la jornada. Para lo mecánico (formateos, renombres, transformaciones repetitivas), el barato: inteligencia de sobra para tareas que no piensan.

La lógica es de cartera: gastas inteligencia cara solo donde equivocarse cuesta caro. Es la misma lógica que aplicas con personas senior y junior; aquí la diferencia es que cambiar de junior a senior cuesta dos clics.

Qué significa que un modelo «razona»

Sube al asiento del copiloto un momento. Cuando le haces una pregunta difícil a un modelo moderno, antes de responder arma internamente una cadena de pensamiento: descompone el problema, prueba caminos, se corrige. Eso es el razonamiento, y se paga en las tres monedas del capítulo 4: tokens (la cadena consume salida), tiempo (piensa antes de hablar) y cuota (más tokens, más créditos).

La metáfora útil: es el modelo haciéndose el examen en borrador antes de entregarlo. Más borrador suele significar mejor respuesta en problemas difíciles; en problemas fáciles, el borrador es desperdicio puro.

La perilla: cuánto pensamiento compras

Ese borrador tiene perilla, y se llama esfuerzo de razonamiento (reasoning effort): un parámetro que controla cuánta cadena de pensamiento compra el modelo antes de contestar (Z.ai, 2026d).

En GLM-5.2 la perilla es completa: eliges desde razonamiento apagado hasta máximo. En GLM-5.3 la filosofía cambió: el razonamiento va siempre encendido (no se puede apagar) y solo eliges la profundidad, con niveles bajo, alto y máximo (Z.ai, 2026d). La idea es que pensar es siempre parte del trabajo; tú solo decides cuánto.

Los números que importan para tu semana: el nivel máximo consume cerca de 3 veces más cuota que el rápido, y el nivel alto conserva casi toda la calidad con menos de la mitad de los tokens (Valverde, 2026). Con eso, mi regla sale sola, y es la misma desde hace meses: máximo para diseño y bugs retorcidos; alto para todo lo demás.

Detalle de campo que confirma que esto es infraestructura y no marketing: en ZCode eliges el esfuerzo con un selector en la interfaz; en OpenCode, a octubre de 2026, el selector para GLM aún no existe y hay una solicitud abierta en su repositorio desde junio (Anomaly, 2026). Cuando una perilla importa, su ausencia se nota.

Preguntas de reflexión

1. De tu semana laboral, ¿qué proporción es explorar, implementar y mecanizar? Reparte tres modelos en esas fases y compara el costo con usar uno solo para todo.

2. Piensa en el último bug retorcido que resolviste. ¿Cuánto de la solución salió de pensar el problema antes de tocar código (el «borrador» del modelo) y cuánto de puro tecleo? ¿Qué nivel de esfuerzo hubieras comprado para esa sesión?

3. Tu herramienta actual, ¿expone la perilla de esfuerzo? Si no, ¿cuánto estás pagando en cuota por razonamiento que no decidiste comprar?

Resumen de puntos clave

– El GLM Coding Plan (desde $18 USD/mes) funciona en 20+ herramientas; créditos con ventanas de 5 horas y semanal, mitad de precio fuera de hora pico.

– El abanico a octubre de 2026: 5.3 insignia (1M, siempre razona), 5.3-Flash (multimodal, 3x más barata de cuota), 5.2 (ajustable), 5.1/Turbo (mecánica).

– El patrón es repartir por fase: contexto gigante para explorar, balanceado para implementar, barato para mecanizar; inteligencia cara solo donde equivocarse cuesta caro.

– Razonar es la cadena de pensamiento previa a la respuesta; se paga en tokens, tiempo y cuota.

– La perilla (reasoning effort): en GLM-5.3, bajo/alto/máximo con el razonamiento siempre encendido; máximo consume ~3x más; mi regla: máximo para diseño y bugs retorcidos, alto para lo demás.

Glosario

– **GLM Coding Plan**: suscripción de Z.ai que da acceso a los modelos GLM dentro de más de 20 herramientas de programación.

– **Razonamiento (cadena de pensamiento)**: trabajo interno del modelo antes de responder: descompone, prueba, se corrige; consume tokens y tiempo.

– **Esfuerzo de razonamiento (reasoning effort)**: parámetro que controla cuánta cadena de pensamiento comprar; niveles bajo/alto/máximo en la familia 5.3.

– **MoE** (mixture of experts): arquitectura con muchos parámetros totales y pocos activos por token; el truco detrás de que la Flash sea barata (320 mil millones de parámetros totales, 18 mil millones activos por token).

Ejercicio

Toma una tarea real de tu lista y cotízala en la tabla de fases: escribe qué fase es (explorar, implementar, mecanizar), qué modelo le pondrías y qué nivel de esfuerzo le comprarías, antes de empezar. Al terminar la tarea, compara: ¿la fase resultó ser la que pensabas, y el esfuerzo que compraste fue el necesario o pagaste borrador de más? Esa comparación, repetida una semana, calibra tu perilla mejor que cualquier benchmark.

Capítulo 9. Memoria entre sesiones: Engram

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Explicar el problema que resuelve la memoria persistente (y por qué el modelo no puede resolverlo solo).
2. Decidir qué merece guardarse en memoria y qué no, usando los cuatro tipos que de verdad se recuperan.
3. Instalar el hábito de arranque: memoria consultada antes de trabajar, no después del desastre.

El amnésico con el que trabajas todos los días

Cierra la sesión y todo muere. La arquitectura que explicaste el lunes, la convención de commits que acordaste, el bug que te comió dos horas y su causa raíz: para el agente de la sesión nueva, nada de eso existe. Tienes que re-explicarlo, y con agentes que participan de verdad en tu trabajo, ese impuesto se paga carísimo (Valverde, 2026).

El capítulo 4 ya te dijo por qué no hay atajo: el modelo es stateless, sin memoria entre llamadas; cualquier «recuerdo» tiene que vivir afuera e inyectarse cuando hace falta. La memoria persistente es exactamente eso, hecho bien: una base donde el agente guarda lo importante y la consulta antes de empezar.

Qué entra y qué no

No todo merece memoria; guardar de todo convierte la base en un pajar donde nadie encuentra agujas. Lo que de verdad se recupera y se paga son cuatro tipos:

Decisiones con su porqué. «Elegimos IndexedDB y no localStorage porque X.» Sin el porqué, en tres meses el agente te propondrá lo descartado, con entusiasmo.

Bugs con causa raíz. El síntoma, la causa y el arreglo. Es el tipo que más ahorra: los bugs enterrados tienden a volver, y la memoria les pone una lápida con instrucciones.

Convenciones del proyecto. Lo que ya está en tu `AGENTS.md` no se duplica aquí (reglas es «cómo trabajamos»; memoria es «qué nos pasó»), pero la convención que nació de un incidente sí: el porqué vive en memoria, la regla en el archivo.

Planes en curso. En qué quedamos ayer, qué falta, qué se decidió pendiente. Es lo que permite que la sesión de hoy empiece corriendo.

Engram, por dentro

Mi memoria es Engram: un binario open source (MIT) escrito en Go, agnóstico del agente, con una base SQLite local con búsqueda de texto completo, que se conecta por MCP y sirve también API, línea de comandos y terminal interactiva (Gentleman Programming, 2026). Traducción práctica: la memoria vive en mi disco, dentro de mi máquina, y cualquier agente que hable MCP puede leerla y escribirle.

Lo agnóstico es la parte estratégica: la memoria no queda secuestrada en la app donde la escribiste. Cambio de herramienta mañana, y el archivo de lo aprendido se viene conmigo. Es la misma lógica de portabilidad del capítulo 7 (reglas) y reaparecerá con las skills del capítulo 11: en este stack, lo que aprendes te pertenece a ti, no a la plataforma.

Qué pasa al arrancar una sesión

El flujo que hace que la memoria pague: al empezar, el agente busca lo relevante (el proyecto, el tema, la sesión previa) y retoma el hilo donde lo dejamos. No es magia: es una búsqueda sobre la base, inyectada al contexto. Y tampoco es opcional en mi flujo: la consulta va antes de trabajar, igual que tú leerías las notas de la reunión anterior antes de opinar.

El ejemplo que cierra el caso, con su historia completa. Mi laptop tiene 16 GB de RAM. Hace un tiempo, un agente lanzó la suite de tests con un worker por núcleo (que es el default que a él le parece razonable) y la máquina se congeló hasta el reinicio. La regla quedó en el `AGENTS.md` (capítulo 7) y el suceso, con su causa, en la memoria. Desde entonces, ningún agente nuevo la vuelve a intentar: la lee antes de trabajar. Nunca tuve que volver a explicarlo (Valverde, 2026). Ese «nunca tuve que volver» es toda la promesa de este capítulo en cuatro palabras.

Preguntas de reflexión

1. De lo que explicaste la semana pasada a una IA, ¿qué merecería lápida en memoria (decisión, bug, convención, plan) y qué era ruido? ¿Qué proporción fue cada cosa?

2. Tu proyecto actual: ¿qué es «cómo trabajamos» (va en reglas) y qué es «qué nos pasó» (va en memoria)? Revisa si tienes datos de la segunda categoría viviendo en la primera.

3. Si mañana cambiaras de app de agentes, ¿qué te llevarías puesto? ¿Cuánto de lo que tu agente «sabe» vive en archivos tuyos y cuánto en la plataforma?

Resumen de puntos clave

– La sesión nueva amanece sin memoria porque el modelo es stateless; la memoria persistente vive afuera y se inyecta antes de trabajar.

– Guardan la renta cuatro tipos: decisiones con porqué, bugs con causa raíz, convenciones nacidas de incidentes y planes en curso; lo demás es pajar.

– Engram: binario Go open source, SQLite local con búsqueda completa, servido por MCP; agnóstico del agente, así que lo aprendido te pertenece.

– El hábito: consultar al arranque, como leerías las notas de la reunión anterior.

– El caso de la laptop de 16 GB: regla en `AGENTS.md`, suceso con causa en memoria, y ningún agente nuevo la vuelve a intentar.

Glosario

– **Memoria persistente**: base externa al modelo donde el agente guarda y de la que recupera entre sesiones.

– **Causa raíz**: el porqué profundo de un bug (no el síntoma); lo que convierte un arreglo en conocimiento reutilizable.

– **SQLite**: base de datos en un solo archivo, sin servidor; donde vive localmente la memoria de Engram.

– **FTS5 (búsqueda de texto completo)**: índice que permite buscar por palabras dentro de la base; lo que hace útil la memoria al arranque.

Ejercicio

Elige el bug más reciente que te haya costado más de una hora. Escríbelo en formato lápida: síntoma (qué se veía), causa raíz (por qué pasaba de verdad) y arreglo (qué lo mató). Si usas Engram u otra memoria, guárdalo ahí; si no, guárdalo donde sea, pero escríbelo. La próxima vez que un síntoma parecido aparezca, mide cuánto tarda la sesión con lápida contra la que la escribió: esa diferencia, sumada por meses, es el argumento completo de este capítulo.

Capítulo 10. MCP: el conector universal

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Explicar qué es un protocolo de herramientas (MCP) desde cero: el problema que estandariza y el modelo mental de servidor como plugin.
2. Evaluar qué conexiones merecen ser servidores en tu stack (y cuáles no) con un criterio simple.
3. Montar el flujo del capítulo: ticket → código → pull request sin salir de tu terminal.

El problema de las integraciones a la medida

Antes de MCP, darle una habilidad nueva a tu agente era integración a mano: código pegado al caso, distinto para cada herramienta, y la mitad rota a la primera actualización. Querías que tu agente abriera tickets de Jira, y dependías de que la app de turno tuviera su plugin de Jira, o de escribir tú el puente.

Los protocolos existen exactamente para eso: acordar cómo hablan dos programas para que cualquiera que hable el idioma se entienda con cualquiera. Piensa en el USB: antes de los estándares, cada dispositivo traía su cable; después, un solo puerto para todo. MCP es el USB de las herramientas del agente.

Qué es MCP

MCP (Model Context Protocol) es un estándar abierto para conectar herramientas al agente, anunciado por Anthropic en noviembre de 2024 y adoptado después por todo el ecosistema (Anthropic, 2024b). La pieza unitaria es el servidor MCP: un programa que le da al agente habilidades nuevas (abrir tickets, consultar documentación, manipular archivos de diseño) y que la app monta como quien monta un plugin.

La diferencia práctica es la que dice la fuente y repito porque no exagera: es la diferencia entre un agente genérico y *tu* agente (Valverde, 2026). El agente que viene de fábrica sabe leer archivos y correr comandos; el tuyo sabe además qué dice el ticket 472 y qué versión de la librería estás usando.

Los tres que no le quito

De mi configuración, los tres servidores que justifican el capítulo:

Atlassian Jira (el oficial). El agente abre el ticket, lee los criterios de aceptación, implementa y mueve el issue de estado. Atlassian publica su servidor remoto con autorización por OAuth, así que no hay llaves que pegar a mano (Atlassian, 2025). El efecto es de fondo: el ticket deja de ser un documento paralelo que copias a mano y se vuelve la interfaz del trabajo. En mi proyecto laboral, el flujo ticket → código → pull request ya casi no me saca de la terminal.

Context7. Documentación vigente de librerías, específica por versión, inyectada al contexto del agente justo cuando la necesita (Upstash, 2025). El clásico desastre del capítulo 4 (la IA que te sugiere una API removida hace dos años, con total seguridad) desapareció de mi flujo el día que lo conecté: el agente consulta la fuente viva en lugar de su memoria de entrenamiento.

Engram. La memoria del capítulo anterior vive aquí: un servidor MCP que el agente consulta al arranque y al que escribe lo aprendido (Gentleman Programming, 2026). El patrón se repite: una habilidad nueva, un servidor, disponible para cualquier app que hable el idioma.

Cómo se conecta en la práctica

Montar un servidor es configuración, no código: cada app tiene su archivo o su pantalla donde declaras la lista de servidores (los remotos con su URL y permiso, los locales con su comando). La parte que sí es tuya es el criterio: qué merece ser servidor.

Mi filtro, en una línea: lo que consultas seguido y estructurado (tickets, docs, memoria) merece servidor; lo que usas una vez al mes y a medias, mejor a mano. Cada servidor suma contexto, permisos y superficie de falla; la colección crece por necesidad, no por catálogo.

Preguntas de reflexión

1. Haz la lista de las cinco cosas que más veces al día copias y pegas entre tu agente y otra herramienta (tickets, docs, tableros, correo). Esa lista es tu cola de servidores pendientes; ordénala por frecuencia.

2. El ticket convertido en interfaz del trabajo cambia dónde vive la verdad del requisito. ¿Qué se gana y qué se pierde cuando el agente escribe directo al ticket?

3. ¿Qué herramienta tuya tiene API pero «no vale la pena» conectar? ¿Qué tendría que pasar (frecuencia, dolor) para que sí valiera?

Resumen de puntos clave

– MCP (Model Context Protocol): estándar abierto de Anthropic (noviembre de 2024) para conectar herramientas al agente; el servidor MCP es el plugin de habilidades.

– El modelo mental: el USB de las herramientas; un idioma que hablan todas las apps y todos los servidores.

– Los tres que se quedan: Jira oficial (el ticket como interfaz del trabajo), Context7 (documentación viva por versión) y Engram (memoria).

– Montar es configuración; el criterio es tuyo: lo frecuente y estructurado merece servidor, lo ocasional se queda a mano.

– La colección crece por necesidad: cada servidor suma contexto, permisos y superficie de falla.

Glosario

– **Protocolo**: acuerdo de cómo se comunican dos programas; lo que convierte integraciones a la medida en estándar.

– **Servidor MCP**: programa que le da habilidades nuevas al agente (tickets, docs, memoria) y se monta como plugin.

– **OAuth**: mecanismo para autorizar a una app a actuar en tu nombre sin entregarle tu contraseña; así se conecta el MCP de Atlassian.

– **Servidor remoto vs. local**: el remoto vive en la nube del proveedor (solo URL y permiso); el local corre en tu máquina (comando propio, como Engram).

Ejercicio

Diseña tu primera conexión sin instalar nada: elige el candidato #1 de tu lista de copiar-pegar (del ejercicio de reflexión 1) y especifica qué tres acciones debería poder hacer el agente (leer X, crear Y, actualizar Z) y qué información le hace falta para cada una. Esa especificación de tres renglones es la mitad del trabajo; la otra mitad es buscar si ya existe el servidor (probablemente sí) y montarlo. Guárdatela: el capítulo 15 la pedirá.

Capítulo 11. Skills: oficios que le enseñas al agente

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Explicar qué es una skill y cómo se diferencia de una regla (`AGENTS.md`) y de un servidor MCP.
2. Elegir skills con criterio de retorno, en vez de coleccionarlas por catálogo.
3. Escribir la primera skill propia: un oficio tuyo, en un archivo portable.

Datos contra oficios

Un modelo trae datos de fábrica: sabe sintaxis, patrones, la historia pública de la programación. Lo que no trae es tu oficio: cómo se hace una buena crítica de interfaz, cómo se arma un examen universitario decente, cómo se debugging un despliegue sin quemar la tarde. Eso se le enseña.

El mecanismo de enseñanza se llama skill: un archivo de markdown con instrucciones para hacer *bien* un tipo de trabajo específico. El agente la carga automáticamente cuando tu tarea coincide con su tema, la lee completa, y trabaja siguiéndola (Valverde, 2026). Piénsalo como enseñarle oficios, y no solo datos: la diferencia entre saber qué es una buena UI y saber auditar una (Valverde, 2026).

Conviene ubicarla contra sus primos, porque el ecosistema confunde: la regla del capítulo 7 dice cómo trabajas tú, siempre, en todo; la skill dice cómo se hace un oficio concreto, cuando toca; el servidor MCP del capítulo 10 da habilidades de conexión (abrir el ticket, leer la doc). Regla = constitución; skill = manual del oficio; servidor = manos.

Casi todas son ruido; tres pagan la renta

Tengo alrededor de 130 skills instaladas, y te lo digo con la misma honestidad de la tabla del capítulo 6: la mayoría no la uso nunca. El número impresiona; el retorno viene de pocas. Las tres que de verdad trabajan en mi stack (Valverde, 2026):

impeccable, crítica de interfaces con criterio profesional. Le paso mi app y me regresa un audit serio contra buenas prácticas. El caso que la vendió: la corrí contra DespensApp (mi app personal de despensa, de la que hablo en el capítulo 14) y encontró cinco problemas reales de usabilidad que yo llevaba meses mirando sin ver. Los meses son la unidad correcta: uno se ciega con su propia pantalla.

document-skills, generación de documentos de oficina de verdad: Word, PowerPoint, Excel y PDF con formato profesional. Los materiales de mi curso universitario (exámenes, presentaciones, listas) salen de aquí a partir de mis notas en markdown. Lo que antes me comía un domingo ahora es una tarde; ese caso abre el capítulo 14, donde lo cuento completo.

superpowers, disciplina de proceso: desarrollo guiado por tests, debugging por causa raíz, planes antes de actuar. Es un proyecto público y popular, una metodología completa empaquetada como skills componibles (Vincent, 2025). Es también el puente a la Parte III: cuando llegue el capítulo 12 (cómo dirijo el trabajo), la disciplina ya estará instalada.

La carpeta portátil: tu método te pertenece

Cada app busca sus skills en una carpeta distinta. La tentación es dejar que cada una tenga su copia; el error aparece el día que actualizas una skill en una app y las demás se quedan viejas. Mi arreglo: una sola carpeta de skills portables que copio (o ligo) a cada app que uso (Valverde, 2026). Un solo lugar para mantener, copiar y listo.

La carpeta portátil es una póliza, igual que la memoria agnóstica del capítulo 9 y las reglas del 7: el oficio que le enseñaste a tu agente es método tuyo, comprado con tu experiencia, y no debería morir con la plataforma que lo cargó ese semestre. En mi stack, todo lo que se aprende vive en archivos míos.

Escribe la tuya

La primera skill propia sale en una tarde, y es la mejor forma de entender el formato. La receta mínima: elige un oficio que hagas seguido y que ya hagas bien (auditar tu UI, armar tu changelog, revisar tus PRs); escribe el procedimiento como se lo explicarías a un junior buenísimo el primer día (pasos, criterios, los errores típicos); guárdalo como markdown con una primera línea que diga cuándo aplica. Listo: la próxima vez que tu tarea toque ese tema, el agente la encontrará y trabajará como tú le enseñaste.

El salto de calidad llega cuando la ofreces al mundo: las skills son archivos de texto, así que la tuya puede vivir en un repositorio y mejorar con uso ajeno, como le pasó a superpowers. Enseñar el oficio a tu agente y enseñárselo al mundo es el mismo archivo (Vincent, 2025).

Preguntas de reflexión

1. Nombra dos oficios que solo tú sabes hacer bien en tu contexto (nadie fuera de tu equipo los haría igual). ¿Qué tendría una skill para cada uno, y qué parte no se puede escribir?
2. De las herramientas que usas por catálogo (porque las recomendó alguien), ¿cuántas usaste más de tres veces? Esa tasa es tu filtro natural para instalar la siguiente.
3. ¿Qué skill tuya terminaría en tu carpeta portátil el primer día, y por qué te daría miedo que solo viviera dentro de una app?

Resumen de puntos clave

– Una skill es un archivo markdown que le enseña al agente un oficio concreto; se carga sola cuando tu tarea coincide con su tema.

– Las tres piezas no compiten: regla (cómo trabajas tú, siempre), skill (cómo se hace un oficio, cuando toca), servidor MCP (manos para conectarse a herramientas).

– De ~130 instaladas, tres pagan la renta: impeccable (crítica de UI; cinco problemas que no veía), document-skills (documentos de verdad; el curso en una tarde) y superpowers (disciplina de proceso).

– La carpeta portátil es la póliza: un solo lugar para mantener, copiar a cada app; el método te pertenece.

– Escribir la tuya: un oficio que dominas, explicado a un junior buenísimo, en un archivo con una línea que diga cuándo aplica.

Glosario

– **Skill**: archivo de instrucciones para hacer bien un tipo de trabajo específico; el agente lo carga cuando la tarea coincide con su tema.

– **Carpeta portable**: carpeta única de skills que se copia o liga a cada app; evita copias divergentes y muertes de plataforma.

– **Audit (de UI)**: revisión sistemática de una interfaz contra buenas prácticas; el oficio de la skill impeccable.

– **Skills componibles**: skills diseñadas para combinarse entre sí y formar metodologías (el enfoque de superpowers).

Ejercicio

Escribe tu primera skill ahora, con la receta del capítulo: el oficio que más repites y mejor dominas, en un archivo markdown cuyo primer renglón diga cuándo aplica («Cuando te pida auditar una pantalla…», «Cuando arme el changelog…»). Incluye pasos, criterios de calidad y los dos errores típicos que cometiste tú antes de aprender. Pruébala esta semana con una tarea real, y anota qué línea de la skill hizo la diferencia; esa línea es tu primer patrón de diseño de skills.

Parte III. Dirigir el trabajo

Capítulo 12. SDD, sub-agentes y tests como árbitro

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Explicar desde cero qué es el desarrollo dirigido por tests (TDD) y por qué con agentes pasa de buena práctica a control de calidad obligatorio.
2. Describir el flujo SDD (spec-driven development) fase por fase, y por qué cada fase la ejecuta un sub-agente distinto con contexto fresco.
3. Decidir cuándo un cambio merece el proceso completo y cuándo la escalera del capítulo 7 basta.

El anti-patrón con el que todos empezamos

Le dices al agente «agrega el login» y rezas. Sale algo, lo pruebas, funciona a medias, le pides correcciones, sale otra cosa, y tres horas después tienes un login que ni entiendes ni te atreves a tocar. El problema no fue el modelo: fue darle la implementación entera en una sola frase, sin especificación, sin verificación, sin dirección.

La Parte III entera trata de lo contrario: dirigir. Este capítulo instala las dos piezas que hacen viable dirigir a escala: un proceso por fases (SDD) y un árbitro que no negocia (los tests).

TDD desde cero: el test que falla primero

Desarrollo dirigido por tests (TDD, test-driven development): antes de escribir la función, escribes la prueba que demuestra que funciona; corre, falla (porque la función no existe); luego escribes el código mínimo que la hace pasar; corre, pasa. Rojo, verde, y a repetir con el siguiente pedazo.

Si nunca lo practicaste, suena a burocracia. Su punto real: el test es una especificación ejecutable. Dice «esto debe pasar» en un lenguaje que ni tú ni el agente pueden interpretar a conveniencia, y el veredicto lo da la máquina, no el ánimo de nadie.

Y aquí va la frase que me cambió la cabeza, tal cual: cuando quien escribe el código es un agente, los tests dejan de ser trámite y se vuelven tu control de calidad automático. El agente propone; los tests deciden si pasa (Valverde, 2026). Con un junior humano había margen para confiar por carisma; con el agente el carisma sobra (y le sobra a él: te diría que sí con esa seguridad del capítulo 4). En modo estricto, la regla es todavía más dura: el agente no implementa nada sin antes escribir el test que falla.

SDD: el cambio grande por fases

Para algo de sustancia («agrega el login», «migra el módulo de pagos»), el flujo SDD (spec-driven development) parte el trabajo en fases, y cada fase produce el insumo de la siguiente (Valverde, 2026):

1. **Explorar.** El agente lee el codebase y regresa un mapa de lo que hay (con el modelo de contexto gigante del capítulo 8, de un bocado).
2. **Proponer.** Sobre el mapa, un plan de ataque con alternativas y riesgos.
3. **Especificar.** La propuesta se convierte en una especificación: qué se hará, qué no, con qué criterios de aceptación.
4. **Diseñar.** La técnica: componentes, contratos, migraciones.
5. **Desglosar en tareas.** La spec en pedazos que caben en una sesión, en orden.
6. **Implementar.** Sub-agente por tarea, con TDD.
7. **Verificar.** Contra la especificación, no contra la esperanza; y cerrar.

La palabra que carga el flujo es la primera: especificación. Todo lo que sigue le debe fidelidad a ese documento, y la verificación final pregunta «¿cumple la spec?», no «¿se ve bien?».

Por qué cada fase la ejecuta un agente distinto

Podrías pedirle a un solo agente que corra el flujo completo en una sesión eterna. Sería un error, y la razón es mecánica: el contexto lleno degrada las decisiones (Valverde, 2026). Un agente que ya leyó cincuenta archivos, tres propuestas y media spec deja de ver el bosque: le sobra pasado y le falta atención. Cada fase arranca con contexto fresco: solo lo que su fase necesita.

La segunda razón es la regla de oro del flujo: quien implementa no debería ser quien revisa (Valverde, 2026). El autor no ve sus propios huecos (tú tampoco los ves en tu código; es humano). Por eso, antes de cada pull request corren revisores frescos especializados: uno de seguridad, uno de legibilidad, uno de tests, uno de resiliencia. Es code review, pero sin la cortesía: cada uno con su lista, contra el diff, sin saber quién escribió qué. El patrón es el mismo que Anthropic describió para sistemas multiagente (orquestador que reparte, trabajadores con contexto propio); aquí el orquestador es el flujo mismo (Anthropic, 2024a).

Los commits chicos, o el arte de deshacer

De propina, el flujo deja una disciplina de cierres: cada unidad de trabajo termina en un commit chico, con inicio claro, fin claro, verificación dicha y camino de rollback (Valverde, 2026). Cuando algo sale mal (y a veces sale), deshacer es trivial: vuelves al commit anterior y listo, en vez de arqueología inversa sobre un diff de mil líneas.

El flujo completo, en producción

Para verlo entero, mi monorepo laboral (API en NestJS, frontends en Next; los nombres se quedan fuera por confidencialidad, el flujo no): llega el ticket de Jira; el agente lo abre por MCP (capítulo 10), lee los criterios de aceptación y explora el código; la propuesta se vuelve spec; los sub-agentes implementan por tareas con TDD; los revisores frescos miran el pull request; los tests corrían en cada paso y el issue se transiciona solo cuando todo pasa. Mi trabajo en ese circuito: decidir la spec, dirimir lo que los revisores discuten, y aprobar. El capítulo 14 vuelve a este terreno para mostrar qué pieza brilla en cuál parte.

Cuándo sí y cuándo no

El flujo completo es para cambios de sustancia. Para lo demás está la escalera del capítulo 7: la solución más corta que funciona, y ya. La pregunta de filtro cabe en una línea: ¿este cambio merece especificación? Si sí, SDD; si no, escalera. La gracia de un proceso pesado es usarlo poco.

Preguntas de reflexión

1. La última característica que le pediste a una IA de un solo prompt: ¿qué había en ella que merecía spec, y qué te costó no tenerla?

2. «El contexto lleno degrada las decisiones»: ¿te ha pasado a ti, humano, en una tarde larga? ¿Qué haces contigo que podrías copiar para tus agentes (descanso, cambio de tarea, contexto fresco)?

3. En tu equipo actual, ¿quién revisa al que implementa? ¿Qué pasaría con la calidad si esa revisión la hiciera alguien sin cortesía y sin relación contigo?

Resumen de puntos clave

– TDD: el test que falla primero; el test es especificación ejecutable y el veredicto lo da la máquina.

– Con agentes, los tests son el árbitro: el agente propone, los tests deciden; en modo estricto, nada se implementa sin test previo.

– SDD por fases: explorar → proponer → especificar → diseñar → desglosar → implementar → verificar; la especificación es la constitución del cambio.

– Cada fase, un sub-agente con contexto fresco (el contexto lleno degrada las decisiones), y quien implementa no revisa: revisores frescos especializados antes de cada PR.

– Commits chicos con rollback claro; el proceso completo solo para cambios de sustancia, la escalera para lo demás.

Glosario

– **TDD** (test-driven development): escribir primero el test que falla y luego el código mínimo que lo pasa; rojo, verde, repetir.

– **SDD** (spec-driven development): partir el cambio grande en fases que producen especificación, diseño y tareas antes de tocar código.

– **Sub-agente**: agente lanzado para una tarea concreta, con contexto fresco y mandate acotado; no hereda la sesión de quien lo lanza.

– **Revisor fresco**: agente (o persona) que revisa código que no escribió, sin la cortesía del que conoce al autor.

– **Rollback**: camino documentado para deshacer una unidad de trabajo; el lujo de los commits chicos.

Ejercicio

Practica el ciclo completo en un cambio real pero acotado (un endpoint, una función, un componente): escribe primero la especificación de cinco renglones (qué hace, qué no, cómo se acepta), luego pide al agente el test que falla, luego la implementación mínima, y cierra con un commit que diga qué verificaste. Anota dónde te dieron ganas de saltarte un paso; ese paso es exactamente el que el próximo agente te va a cobrar caro.

Capítulo 13. Diseño con IA

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Separar las dos herramientas de diseño del stack (Stitch y Pencil) por lo que de verdad aportan al flujo: inspiración contra loop de producción.
2. Correr el loop completo: pedir → generar → mirar → criticar contra buenas prácticas → corregir.
3. Decidir con honestidad dónde queda cada una (y qué le cambia el rediseño de Stitch 2.0 a esa decisión).

El diseño, la pieza que faltaba en el loop

Todo el libro hasta aquí acelera el código; el diseño era la parte de mi flujo que seguía fuera del agente. Le pedías «hazme una pantalla bonita» y el resultado era esa interfaz genérica que solo sirve para rehacerla. Para que el diseño entrara al loop se necesitaban dos cosas: una herramienta que generara con calidad y otra que dejara corregir con precisión. Usé dos caminos y me quedé con ambos, porque no compiten.

Stitch: la fábrica de inspiración

Stitch (de Google) genera pantallas completas a partir de lenguaje natural: le describes la app y te regresa un diseño presentable en segundos. Es la herramienta que uso para explorar dirección visual al empezar algo: cinco variantes de la misma pantalla en cinco minutos, y de ahí se conversa (Valverde, 2026).

Su límite histórico era la iteración: generaba lindo pero no dejaba ajustar («bájale al contraste de ese botón» no existía en el flujo). Aquí va la actualización honesta, porque este libro se fact-checkea: en marzo de 2026 Google lo rediseñó a fondo como Stitch 2.0, con canvas infinito y mejoras reales de iteración y exportación (Google, 2026). Mi veredicto tras el 2.0: la brecha se cerró bastante, y la que sigue abierta es la de fondo: genera desde modelos propios, sin conocer tu design system ni tus componentes, así que el resultado sigue siendo punto de partida, no destino. Inspiración con criterio; producción, no.

Pencil: el canvas que el agente sí maneja

Pencil (pen.dev) es lo contrario en el eje que importa: una app de diseño cuyo canvas el agente edita directamente vía MCP (capítulo 10), con los archivos de diseño (formato .pen, que por dentro es JSON) viviendo junto al código en tu repositorio (Pencil, 2026). Le pides la pantalla, la diseña en el canvas, y cuando dices «bájale al contraste de ese botón», el agente selecciona ese botón y le baja el contraste. El loop se cierra dentro de tu flujo, sin exportar ni pegar.

Esa diferencia técnica (MCP sobre el canvas) es la que hace al diseñador incansable del que hablo abajo: el agente puede ver el resultado de su propio trabajo y corregirlo, que es exactamente el bucle de observar y verificar del capítulo 5, aplicado a píxeles.

El loop completo, con juez incluido

Así corro diseño hoy, paso a paso:

1. **Pedir.** Describo la pantalla con su propósito (para quién, qué hace).
2. **Generar.** Stitch para explorar dirección si es algo nuevo; Pencil directo si ya tengo la idea.
3. **Mirar.** El agente (o yo) examina el resultado contra la pantalla real.
4. **Criticar.** La skill de crítica de UI del capítulo 11 (impeccable) hace el audit contra buenas prácticas: contraste, jerarquía, tamaños de toque, estados.
5. **Corregir.** El mismo agente corrige en el canvas de Pencil lo que la crítica marcó, y entrega la versión arreglada.

La pieza que menos esperaba disfrutar y la que más uso para prototipos: pedirle una pantalla, que la critique él mismo contra buenas prácticas, y recibir la versión corregida (Valverde, 2026). Un diseñador incansable es la descripción exacta; trabaja a las once de la noche y nunca se ofende con los comentarios.

Qué usar cuándo

Stitch para explorar dirección visual al inicio (varias variantes baratas, con el 2.0 incluso iterando un par de rondas); Pencil para producir el prototipo que el equipo va a tocar y que vivirá junto al código. Si tu semana de diseño es cero pantallas nuevas y puros ajustes, solo Pencil: la fábrica de inspiración no tiene nada que fabricar.

Preguntas de reflexión

1. ¿Qué parte de tu trabajo de diseño (o del que le encargas a alguien) es explorar dirección y qué parte es producir y ajustar? Esa proporción decide cuál herramienta te sirve.
2. El loop del capítulo usa una skill como juez del propio trabajo del agente. ¿Qué otras tareas tuyas tendrían un juez automático de ese tipo, y contra qué criterio mediría?
3. El diseño como archivo junto al código (.pen, JSON) cambia quién puede tocar el diseño. ¿Quién gana esa llave en tu equipo, y qué se rompería?

Resumen de puntos clave

– Dos herramientas, dos papeles: Stitch para inspiración y exploración de dirección; Pencil para el prototipo iterable con el canvas editable por el agente vía MCP.

– Stitch 2.0 (marzo de 2026) cerró buena parte de la brecha de iteración (canvas infinito, mejor export); la brecha de fondo sigue: no conoce tu design system.

– El loop completo: pedir → generar → mirar → criticar con la skill de UI → corregir en el canvas; el juez hace la diferencia entre pantalla bonita y pantalla correcta.

– Los archivos de diseño viven junto al código (.pen, JSON en el repo): diseño versionado, diffs revisables.

– La descripción que se queda: un diseñador incansable, sobre todo para prototipos.

Glosario

– **Stitch**: herramienta de Google Labs que genera pantallas desde lenguaje natural; exploración de dirección visual.

– **Pencil (pen.dev)**: app de diseño con canvas agéntico editable vía MCP; archivos .pen (JSON) en el repositorio.

– **Design system**: el conjunto de componentes y reglas visuales de una marca o producto; lo que los generadores genéricos no conocen.

– **Audit de UI**: revisión sistemática de una interfaz contra buenas prácticas (contraste, jerarquía, accesibilidad); el oficio de la skill crítica.

Ejercicio

Corre el loop en una pantalla tuya esta semana: toma una pantalla existente de tu app (o del proyecto que sea), pídele al agente (con la skill de crítica que tengas, o con una lista de buenas prácticas escrita por ti) un audit honesto, y luego pídele que corrija lo marcado en el canvas de Pencil o directamente en el código. Mide las dos cosas que importan: cuántos problemas encontró el juez que tú ya no veías, y cuánto tardó el ciclo completo. Con esas dos cifras, decide si el diseñador incansable entra a tu flujo fijo.

Capítulo 14. Los campos de prueba

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Ver el stack completo funcionando en tres terrenos reales, con qué pieza brilla en cada uno.
2. Elegir tu propio terreno de arranque con criterio (el más chico donde te des permiso de romper).
3. Reconocer el patrón de adopción que los tres terrenos comparten: empezar chico, medir, crecer.

Tres terrenos, tres estreses distintos

Un stack no se conoce leyendo; se conoce cargándolo. Los míos cargan distinto: uno estresa la velocidad, otro la seriedad, otro el retorno inmediato. Si vas a adoptar piezas de este libro, te conviene ver dónde trabaja cada una a plena carga antes de decidir por dónde entrar tú.

DespensApp: el laboratorio

Mi app personal de despensa (React con Tailwind y IndexedDB, todo local en el navegador). Aquí no hay cliente, no hay hora de entrega, no hay permiso que pedir: es el terreno donde pruebo cosas antes de que se ganen el derecho de llegar al trabajo (Valverde, 2026).

En DespensApp nació el audit de UI del capítulo 13 (la skill encontró cinco problemas de usabilidad que yo llevaba meses mirando sin ver), ahí pruebo las skills de diseño antes de adoptarlas, y ahí vive una suite de 209 tests que corre en segundos. Ese número no es jactancia: es la infraestructura que me deja experimentar sin miedo, porque si un agente rompe algo, el árbitro del capítulo 12 lo grita antes de que yo abra la app. El laboratorio funciona porque tiene alarma.

El monorepo del trabajo

El otro extremo: API en NestJS con frontends en Next, usuarios reales, y confidencialidad de por medio (los nombres se quedan fuera; el flujo no, porque ya lo conté en el capítulo 12). Aquí el stack completo se nota de principio a fin: el ticket de Jira entra por MCP, la spec la dirime el equipo, los sub-agentes implementan con TDD, los revisores frescos miran el pull request, y el issue se transiciona solo cuando todo pasa (Valverde, 2026).

Lo que este terreno enseña y el laboratorio no puede: la disciplina de la memoria (capítulo 9) vale oro cuando el proyecto es grande y la rotación de contextos es constante, y las reglas (capítulo 7) son lo único que impide que cada

agente nuevo proponga su arquitectura favorita en tu código de producción. En el trabajo, el stack no acelera por velocidad; acelera por confianza verificada.

El curso universitario

El terreno más humilde y el de retorno más rápido: los materiales de mi curso. Presentaciones, exámenes, listas de ejercicios: todos salen del agente a partir de mis notas en markdown, con la skill de documentos del capítulo 11 poniendo el formato profesional (Valverde, 2026).

Lo que antes me comía un domingo ahora es una tarde. Es la cifra relativa más honesta que tengo del stack (y es experiencia, no benchmark): una tarea repetitiva, bien especificada por años de hacerla a mano, delegada con la pieza correcta. Cuando alguien me pregunta por dónde empezar, este terreno es mi respuesta: busca tu equivalente del domingo perdido.

Qué pieza brilla dónde

La síntesis cruzada, en prosa: la memoria brilla en el monorepo (proyecto grande, sesiones rotando) y apenas se nota en el curso; las skills brillan en el curso (oficios repetitivos bien definidos) y en DespensApp; MCP brilla en el monorepo (el ticket como interfaz) y en diseño (el canvas de Pencil); la dirección (SDD, tests) solo brilla donde romper duele. Ninguna pieza brilla en todos lados: por eso el capítulo 15 arma el mínimo viable en orden de retorno y no en orden de catálogo.

Empieza donde te des permiso

La lección transversal de los tres terrenos: cada uno empezó chico. DespensApp nació como app de despensa, no como laboratorio; el flujo del monorepo empezó con un ticket; el curso empezó con una presentación. Ningún terreno exige el stack completo el día uno; todos lo fueron ganando pieza por pieza, cuando un dolor concreto la justificó. El permiso te lo das tú, y el terreno correcto para empezar es el más chico donde romper no duela.

Preguntas de reflexión

1. Haz tu propia tabla mental: tus terrenos actuales (trabajo, personales, otros) contra las piezas del stack. ¿Qué pieza tendría el retorno más rápido en cuál terreno?
2. «El laboratorio funciona porque tiene alarma»: ¿tu proyecto personal tiene una? ¿Qué te frena de montarle la más mínima (un test, un lint, un script)?
3. ¿Cuál es tu «domingo perdido», la tarea repetitiva que haces bien y que nadie te paga por hacer lento? Esa es tu primera delegación candidata.

Resumen de puntos clave

– Tres terrenos, tres estreses: DespensApp (laboratorio personal sin permisos, 209 tests como alarma), el monorepo laboral (el flujo completo en producción, confianza verificada) y el curso (retorno más rápido: el domingo que se volvió tarde).

– Cada pieza brilla en su terreno: memoria en el grande, skills en el repetitivo, MCP en el ticket y el canvas, dirección donde romper duele; ninguna en todos lados.

– El patrón de los tres: empezar chico, crecer pieza por pieza cuando un dolor concreto la justifica.

– El terreno de arranque correcto es el más chico donde romper no duela.

Glosario

– **Laboratorio (de stack)**: proyecto personal donde probar piezas sin permiso ni riesgo; funciona si tiene alarma (tests).

– **Monorepo**: repositorio único con varios proyectos adentro (aquí: API y frontends juntos).

– **IndexedDB**: base de datos del navegador; la elección que mantiene a DespensApp 100% local.

– **Cifra relativa**: mejora medida contra tu propia línea base (domingo → tarde), no contra un benchmark externo.

Ejercicio

Elige tu terreno de arranque con las reglas del capítulo: lista tus tres proyectos candidatos (uno laboral, uno personal, uno repetitivo-docente si lo tienes) y califica cada uno en tres renglones: qué duele si se rompe, qué tarea repetitiva tiene, y qué alarma existe hoy. Gana el de menor dolor con mayor repetición. Ese terreno y esa tarea entran al capítulo 15 como tu pieza número uno.

Capítulo 15. Si quieres armar el tuyo

Objetivos de aprendizaje

Al terminar este capítulo podrás:

1. Montar tu stack mínimo viable en orden de retorno, sabiendo qué capítulo releer cuando cada pieza duela.
2. Aplicar la regla de la máquina justa (workers a 2) antes de que tu laptop te la enseñe congelada.
3. Ubicar el camino local (Ollama) como frontera honesta: para qué sirve y por qué este stack no vive ahí.

Del relato al manual

Todo el libro fue relato; este capítulo es manual. La diferencia importa: el relato te mostró mi stack completo, con sus capítulos de contexto (los primeros cinco) y sus piezas (la Parte II) y su dirección (la Parte III); el manual te da el orden de adopción, que es distinto del orden del libro y mucho más corto. Cinco piezas, en orden de retorno y no de moda (Valverde, 2026).

El mínimo viable, pieza por pieza

1. Un agente de código con soporte de MCP. El auto donde vive todo (capítulo 6). Los hay gratis y de pago; elige uno y quédate: el costo de estar cambiando de app supera rápido el de cualquier elección razonable. Que hable MCP es el único requisito duro, porque las piezas 3 y 4 se conectan por ahí.

2. Tu archivo de reglas global. Una tarde bien invertida, el mejor retorno de la lista (capítulo 7): idiomas, convenciones, la escalera, los límites de tu máquina. Si solo haces una pieza hoy, que sea esta: funciona con cualquier agente, incluso con el chat más simple que uses ya.

3. Memoria persistente. Sin ella, todo lo demás se olvida al cerrar la sesión (capítulo 9): decisiones con porqué, bugs con causa raíz, planes en curso. La mía es Engram porque es agnóstica; la tuya puede ser cualquiera que puedas llevarte puesta.

4. Documentación vigente. Context7 o equivalente (capítulo 10): que el agente consulte la fuente viva y no su memoria de entrenamiento. Adiós a la API removida hace dos años sugerida con total seguridad.

5. TDD como gate. Nunca apruebes código del agente sin tests que demuestren que funciona (capítulo 12). Esta es la pieza que convierte la velocidad en confianza: el agente propone, los tests deciden.

Si hiciste los ejercicios del libro, ya tienes el insumo: el diagnóstico de cuatro renglones del capítulo 3 (qué pieza te falta más) y la especificación de conexión del capítulo 10 (tu primer servidor MCP). Este capítulo solo los ordena: empieza

por la pieza número 2 (hoy mismo, una tarde), y las demás por el orden de tu diagnóstico.

La regla de la máquina justa

Antes de soltarte, la regla que aprendí congelando la laptop (la historia completa va en el capítulo 9): si tu máquina es justa de RAM, limita los workers de los tests a 2 (Valverde, 2026). Los agentes aman abrir un hilo por núcleo (a ellos les parece lo eficiente), y una máquina de 16 GB con una suite corriendo a plena paralelidad se congela hasta el reinicio. La regla va en tu `AGENTS.md` y no se discute: es más barata que una laptop nueva.

¿Y todo local?

La pregunta que falta, con su frontera honesta. Ollama es el camino local maduro: un runtime que corre modelos de pesos abiertos en tu propia máquina (el mapa de modelos del capítulo 4), con privacidad total, cero costo por token y cero dependencia de la nube (Ollama, 2023).

Cuándo tiene sentido: cuando la privacidad manda (código que no puede salir de tu red por contrato o por sentido común), cuando no hay conexión estable, o cuando quieres experimentar con modelos sin suscripción. Para esas vidas, Ollama es una buena pieza y el capítulo 4 te dio el mapa para elegir qué correr.

Por qué este stack no vive ahí: mi opinión, fechada a octubre de 2026, y la sostengo con la mecánica del capítulo 4: los modelos locales de hoy cuestan más de lo que dan para trabajo agéntico sostenido (sesiones largas, contexto grande, varios sub-agentes), y el diferencial de calidad contra la nube se paga en cada ciclo del bucle. Lo local brilla en sus casos; el día a día agéntico, por ahora, corre mejor donde la potencia está. Cuando eso cambie, este capítulo se reescribe, y el propio stack será el primero en avisarme.

El cierre que le debe al principio

El capítulo 1 terminó con una pregunta: ¿vas a ser de los que dirige el proceso, o de los que nomás ven qué sale? Los quince capítulos de distancia no la hicieron más pequeña; la hicieron más contestable.

Dirigir, en este mundo, es exactamente lo que armaste leyendo: saber qué es el modelo y qué es el alrededor (Parte I); haber conquistado el alrededor con archivos tuyos: reglas, memoria, contexto, oficios (Parte II); y tener un método para que el trabajo te llegue verificado, con especificación, agentes frescos y tests como árbitro (Parte III). El stack completo es caro de leer y barato de adoptar: cinco piezas, una tarde para empezar, y cada pieza salva su propio dolor cuando toca.

La línea del tiempo del capítulo 2 empezó con un intermediario copiando y pegando errores en un chat. El stack que armaste aquí es la distancia exacta entre aquel octubre y este: la misma persona, el mismo oficio, dirigiendo a un equipo

que no duerme. Que salga algo bueno ya no depende del modelo; depende, como siempre en este oficio, de quien dirige.

Preguntas de reflexión

1. De las cinco piezas del mínimo, ¿cuál es la que ya tienes y la que te falta hace más tiempo? ¿Qué dolor concreto de tu semana desaparecería al cerrar esa brecha?

2. Tu contexto personal (empresa, contrato, país, hardware), ¿empuja hacia la nube o hacia lo local? Escribe la frase que responde y guárdala con tu diagnóstico del capítulo 3.

3. Vuelve a la pregunta del capítulo 1 y respóndela con calendario: ¿qué pieza montas esta semana, cuál este mes, y con qué evidencia sabrás que ya diriges?

Resumen de puntos clave

– El mínimo viable, en orden de retorno: agente con MCP → reglas globales → memoria persistente → documentación vigente → TDD como gate.

– La pieza número 2 (reglas) se monta hoy mismo en una tarde y funciona con cualquier agente, incluso el más simple.

– La regla de la máquina justa: workers de tests a 2 en el `AGENTS.md` de cualquier equipo con RAM justa; más barata que una laptop nueva.

– Lo local (Ollama) es una buena pieza para privacidad, sin conexión y experimentación; a octubre de 2026, el trabajo agéntico sostenido corre mejor en la nube (opinión fechada).

– La tesis devuelta: dirigir es tener el alrededor en archivos tuyos y el método que verifica; el resultado depende de quien dirige, no del modelo.

Glosario

– **Mínimo viable (de stack)**: las cinco piezas que capturan la mayoría del retorno; cada una se adopta cuando su dolor toca.

– **Runtime local**: programa que corre modelos en tu propia máquina; Ollama es el más usado.

– **Worker (de tests)**: proceso paralelo de la suite; en máquinas justas, limitarlos a 2 evita congelamientos.

– **Gate (TDD)**: condición de aprobación: sin tests que demuestren que funciona, no se aprueba código de agente.

Ejercicio

El plan de adopción, en una página: retoma tu diagnóstico del capítulo 3 (las cuatro piezas) y tu terreno ganador del capítulo 14, y escribe el plan de tres pasos: esta semana (pieza 2: tu `AGENTS.md`, y la tarea repetitiva de tu «domingo perdido» encomendada al agente), este mes (pieza 3 y 4 en tu terreno: memoria y documentación), y este trimestre (pieza 5 con TDD en un cambio real de

sustancia). Al final del trimestre, vuelve al capítulo 1 y contéstale la pregunta con evidencia; si llegaste hasta aquí haciendo los ejercicios, ya tienes más stack que el intermediario de octubre de 2025.

Referencias

Anomaly. (2026). *OpenCode*. https://github.com/anomalyco/opencode

Anthropic. (2024). *Building Effective Agents*. https://www.anthropic.com/research/building-effective-agents

Anthropic. (2024). *Introducing the Model Context Protocol*. https://www.anthropic.com/news/model-context-protocol

Atlassian. (2025). *Atlassian Remote MCP Server*. https://mcp.atlassian.com/v1/mcp

Fowler, Martin. (2026, 2 de abril). *Harness engineering for coding agent users*. https://martinfowler.com/articles/harness-engineering.html

Gentleman Programming. (2026). *Engram: persistent memory system for AI coding agents*. GitHub. https://github.com/Gentleman-Programming/engram

Google. (2025, 18 de noviembre). *Introducing Google Antigravity, a new era in AI-assisted development*. https://antigravity.google/blog/introducing-google-antigravity

Google Labs. (2026, 18 de marzo). *Introducing "vibe design" with Stitch*. https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-ai-ui-design

Hugging Face. (2026, 25 de mayo). *Harness, Scaffold, and the AI Agent Terms Worth Getting Right*. https://huggingface.co/blog/agent-glossary

InfoWorld. (2026). *Z.ai disables coding assistant feature after flaw exposed enterprise code upload risk*. https://www.infoworld.com/article/4225022/z-ai-disables-coding-assistant-feature-after-flaw-exposed-enterprise-code-upload-risk.html

Karpathy, Andrej. (2025, 2 de febrero). *Vibe coding (publicación en X)*. X. https://x.com/karpathy/status/1886192184808149383

Ollama. (2023). *Ollama: get up and running with large language models locally*. https://ollama.com

Pencil. (2026). *Pen.dev: an agentic canvas for building bold software*. https://pen.dev/

Upstash. (2025). *Context7: up-to-date docs for AI agents*. https://github.com/upstash/context7

Valverde, Jesús. (2026). *Mi stack de desarrollo con IA*. fuentes/resumen-stack-dev-ia.md

Vincent, Jesse. (2025). *Superpowers: an agentic skills framework*. GitHub. https://github.com/obra/superpowers

Z.ai. (2026). *GLM Coding Plan Documentation*. https://docs.z.ai/devpack

Z.ai. (2026, 26 de agosto). *GLM-5.3-Flash: Frontier Intelligence, Flash Cost*. https://z.ai/blog/glm-5.3-flash

Z.ai. (2026, 14 de agosto). *GLM-5.3: Frontier Coding with Emergent Cyber Capabilities*. https://z.ai/blog/glm-5.3

Z.ai. (2026). *Thinking and reasoning effort (GLM API guide)*. https://docs.z.ai/guides/capabilities/thinking

Z.ai. (2026, 21 de septiembre). *ZCode: Agentic Development Environment*. GitHub. https://github.com/zai-org/ZCode

agents.md. (2025). *Agents.md*. https://agents.md